

EEE3027 Lab report

LTspice

Lab 1

NMOS sweep

```
1. * Parameter initialisation
2. .param VGSVAL=1.2
3. ** VGS connections between ground and a node (gate) set at VGSVAL parameter
4. VGS G 0 DC (VGSVAL)
5. ** VDS connected to ground and drain set at 0V (drain)
6. VDS D 0 DC 0
7. ** NMOS Transistor named 'M1' using an NLEVEL1 model. w = Width
8. *L = length it has drain ,gate, source at ground and bulk at ground.
9. M1 D G 0 0 NLEVEL1 W=10u L=1u
10. ** VTO = Threshold voltage, KP = Transconductance parameter,
11. *LAMBDA = channel-length modulation parameter
12. .model NLEVEL1 NMOS LEVEL=1 VTO=0.4 KP=200u LAMBDA=0.02
13. *0.01 is the steps of the sweep for VDS,
14. *VGS shows that it sweeps in 0.2 steps between 0.4 and 1.2
15. .dc VDS 0 1.2 0.01 VGS 0.4 1.2 0.2
16. ** Plots drain current
17. .plot DC I(M1)
18. .end
19.
```

Figure 1: NMOS DC Sweep netlist

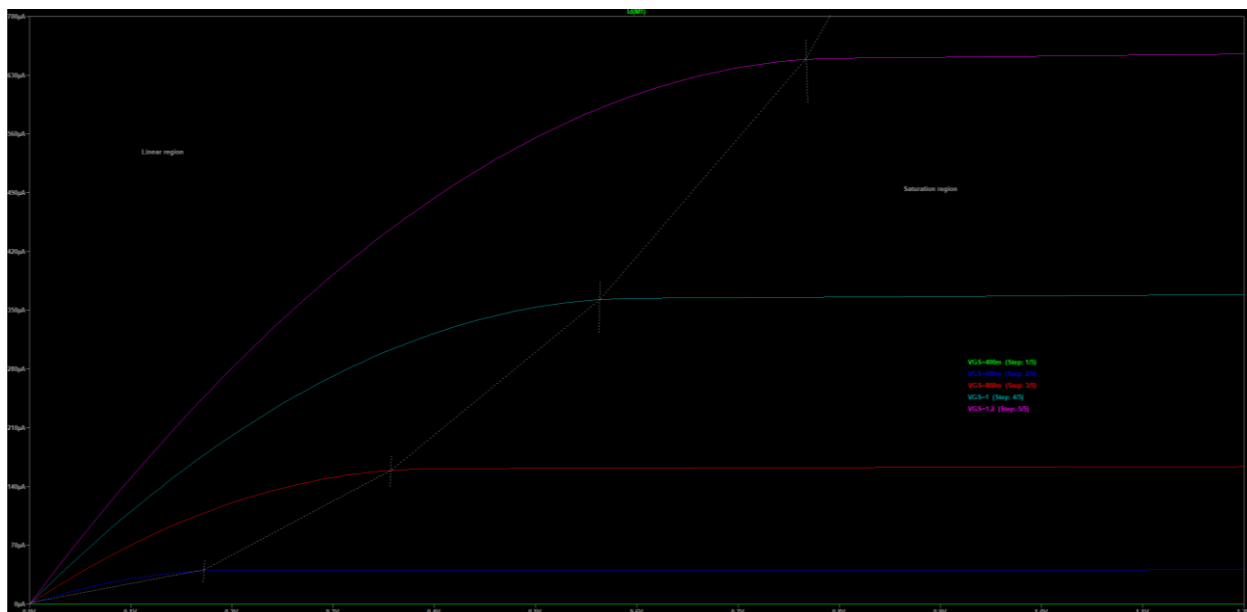


Figure 2: NMOS DC Sweep simulation

The Vds boundary is 0.8 as $V_{TO} = 0.4 = V_{th}$ and $V_{GS} = 1.2$ therefore $1.2 - 0.4 = 0.8$ Vdsn per N-Type transistor characteristics equations. The simulation sweeps Vds 0 to 1.2 and sweeps VGS 0 to 1.2 as changing both values are needed for different lines/steps in the graph. When $V_{GS} - V_{TO} = V_{DS}$ then that is when the saturation region is about to occur.

The linear region is when we can increase gate voltage on the NMOS and see a linear gain on the drain current. This occurs when $V_{gsn} - V_{thn} > V_{dsn}$. The saturation region is when further increasing gate voltage does not influence the drain current because of the NMOS Channel and this occurs when $V_{gsn} - V_{thn} \leq V_{dsn}$.

PMOS

```
1. * Parameter initialization
2. .param VGSVAL=-1.2
3.
4. ** Changed to negative for PMOS
5.
6. ** VGS connections between ground and a node (gate) set at VGSVAL parameter
7. VGS G 0 DC {VGSVAL}
8.
9. ** VDS connected to ground and drain set at 0V (drain)
10. VDS D 0 DC 0
11.
12. ** PMOS Transistor named 'M1' using an PLEVEL1 model. w = Width and L = length
13. ** drain, gate, source at ground and bulk at ground.
14. M1 D G 0 0 PLEVEL1 W=10u L=1u
15.
16. ** Changed to PMOS model
17. ** VTO = Threshold voltage (negative for PMOS), KP = Transconductance parameter, LAMBDA =
channel-length modulation parameter
18. .model PLEVEL1 PMOS LEVEL=1 VTO=-0.4 KP=200u LAMBDA=0.02
19.
20. ** Changed to PMOS with negative VTO
21.
22. ** DC sweep: VDS from 0 to 1.2V in 0.01V steps, VGS from -0.4 to -1.2V in -0.2V steps
23. .dc VDS 0 1.2 0.01 VGS -0.4 -1.2 -0.2
24.
25. ** plots drain current (will be negative for PMOS)
26. .plot DC I(M1)
27. .end
28.
```

Figure 3: PMOS DC Sweep netlist

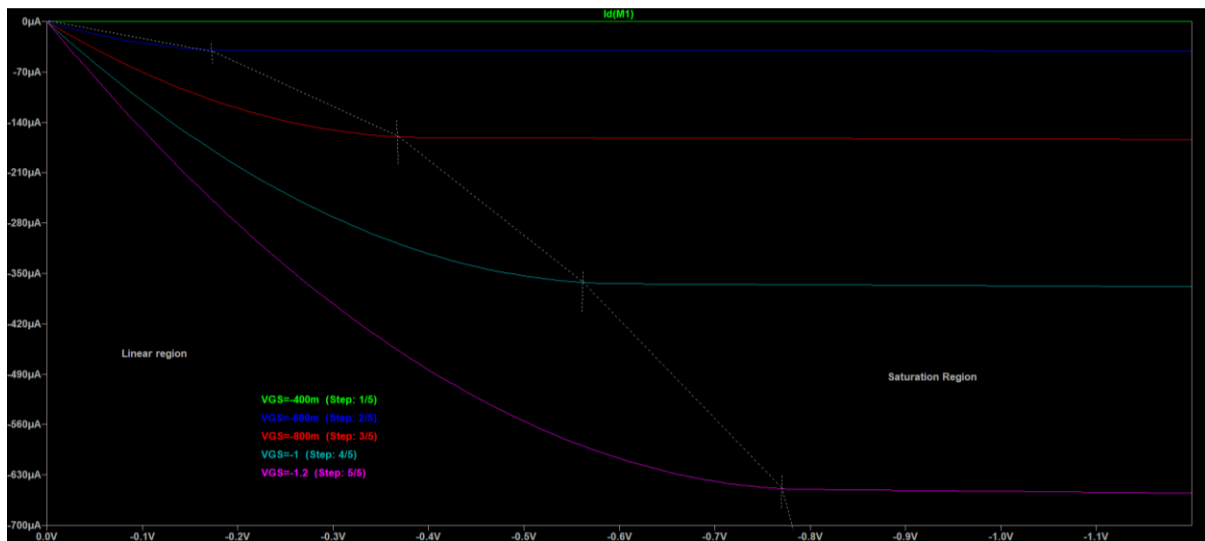


Figure 4: PMOS DC Sweep simulation

Again, figure 4 shows PMOS MOSFET I-V behaviour. The linear region is when $V_{DS} > V_{GS} - V_{TO}$ and the saturation region is when $V_{DS} < V_{GS} - V_{TO}$ (Assuming current are negative values) with the boundary being $V_{DS} = V_{GS} - V_{TO}$ (Noting $V_{TO} = V_{th}$). The linear region is when increasing V_{GS} 's magnitude increases drain current's magnitude in a linear relationship. As soon as it hits the saturation then this relationship stops and increasing V_{GS} 's magnitude does not change the drain current.

Gain-factor consolidation

```

1. * Task 3:  $\beta$  Gain Factor Sweep using .step param
2. * Sweep W from 2 $\mu$ m to 20 $\mu$ m to observe linear scaling of ID
3.
4. .param VGSVAL=1.2 VDSVAL=1.2
5. .param LVAL=1u
6.
7. *Voltage sources to bias gate and drain
8. VGS G 0 DC {VGSVAL}
9. VDS D 0 DC {VDSVAL}
10.
11. * NMOS transistor with swept WVAL
12. M1 D G 0 0 NLEVEL1 W={WVAL} L={LVAL}
13.
14. * Shichman-Hodges Level 1 model
15. .model NLEVEL1 NMOS LEVEL=1 VTO=0.4 KP=200u LAMBDA=0.02
16.
17. * Sweep W from 2 $\mu$ m to 20 $\mu$ m in 2 $\mu$ m steps
18. .step param WVAL 2u 20u 2u
19.
20. * DC operating point analysis
21. .op
22.
23. * Output drain current for each step
24. .print OP V(G) V(D) I(M1) WVAL={WVAL}
25.
26. .end

```

Figure 5: Beta sweep netlist

Figure 5 shows VGS at value 1.2A, VDS at 1.2A and Channel length as 1 micrometre. These are the parameters. VTO is the threshold voltage at zero bias, KP represents Transconductance parameters, Lambda is channel-length modulation parameter. All the equations assume that KP and lambda are constant and do not change.

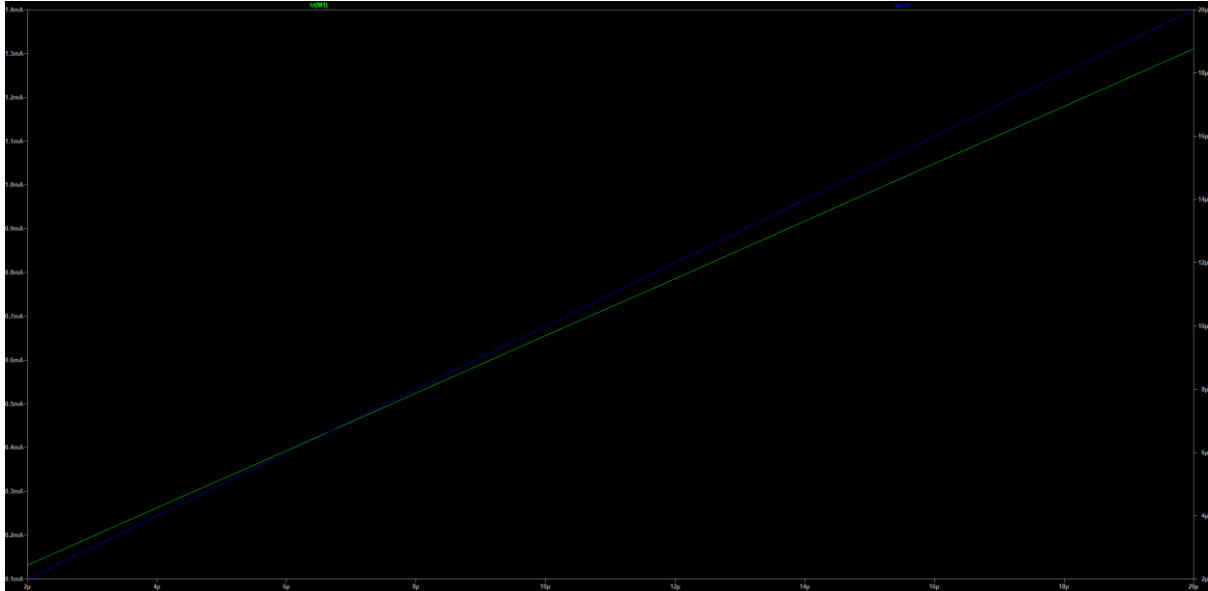


Figure 6: Relationship between width and drain current.

The figure 6 slope is 65.536 (obtained from traces). This simulation makes sense as I_{ds} has a linear relationship with β as seen in figure 6, and $\beta = (WC_{ox} \mu)/L$ so therefore increasing W increases β which increases drain current.

$$I_{ds(\text{linear})} = \beta [(V_{gs} - V_{th}) V_{ds} - \frac{1}{2} V_{ds}^2]$$

$$\text{where } \beta = (WC_{ox} \mu)/L$$

$$I_{ds(\text{sat})} = (\beta/2) (V_{gs} - V_{th})^2$$

Figure 7: Drain current equations

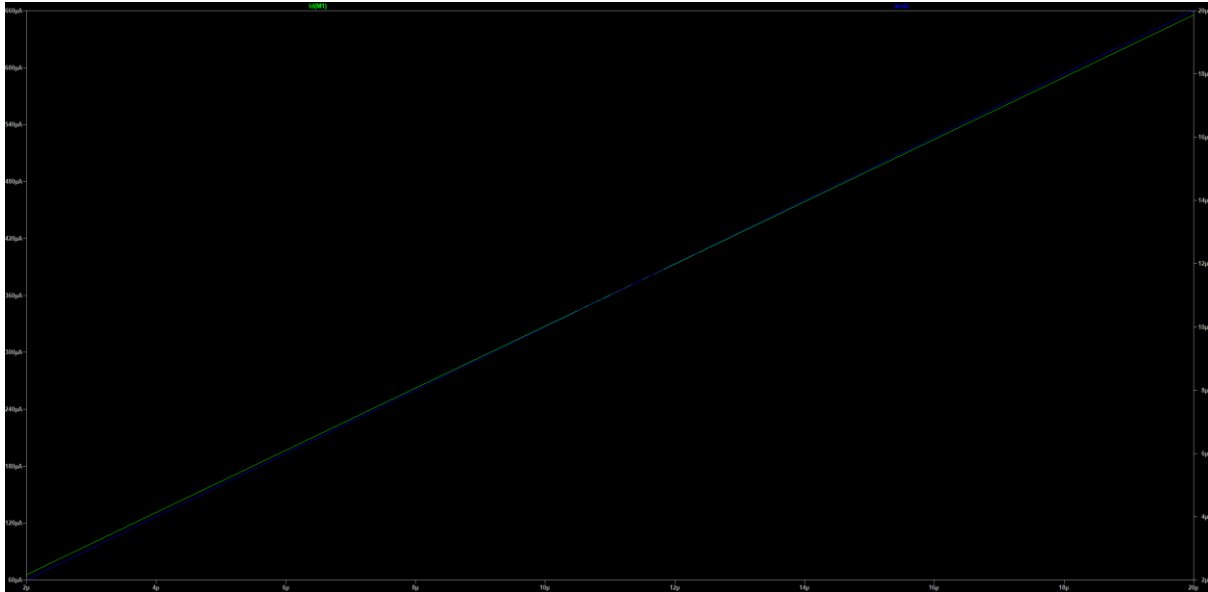


Figure 8: Length doubled in beta sweep

Slope here is 32.768. As L is inversely proportional to β this means that drain current will be halved. Figure 8 peaks at ~ 660 micro amps whilst figure 6 peaks at ~ 1.3 mA showing that simulation is behaving correctly (The slope is exactly half that of figure 6).

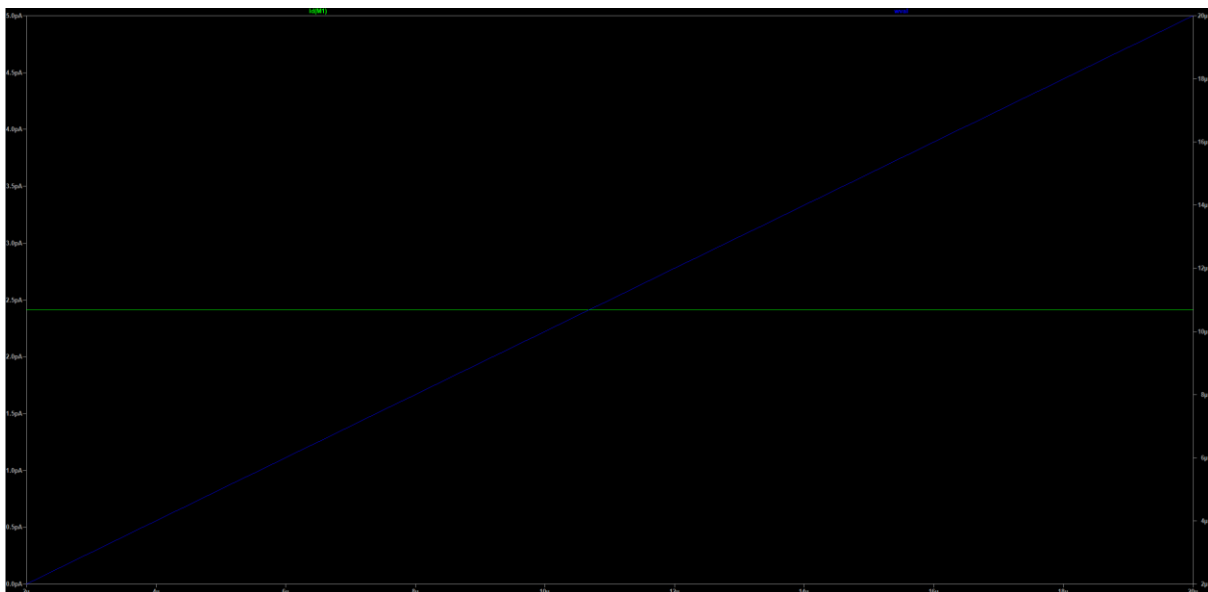


Figure 9: $V_{GS} < V_{th}$ ($V_{GS} = 0.3$, $V_{TO} = 0.4$)

Figure 9 shows that drain current stays the same despite increasing width. This means that the design is in a cutoff region and acts like an open switch. This makes sense as V_{GS} must overcome the threshold to be in the linear region and as the current is in the Pico magnitude means that it can essentially be seen as zero (As $V_{GS} < V_{th} = \text{OFF}$ and $V_{GS} \Rightarrow V_{th} = \text{ON}$ region). V_{GS} being lower than threshold voltage means there is no inversion layer in the MOSFET therefore no current flowing.

PMOS are wider than NMOS due to hole mobility as they are less mobile than electrons, so the added width increase beta so increase drain current to compensate for this.

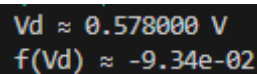
Spice under the hood

Spice solves nonlinear equation iteratively. For example, diodes I-V relationship is non-linear. This iteration means it takes the equation and starts giving sample answers and keeps changing that answer to get as close to 0 as possible. By iteration is by trial and error but in formatted steps increasing or decreasing. This is an example of why SPICE needs iterative solving, as many components will have non-linear relationships so therefore needs a fast computation to solve equations.

```
1. import math
2.
3. # Parameters given
4. Vs = 5.0
5. R = 1000.0
6. Is = 1e-12
7. Vt = 0.026
8. n = 1.0
9.
10. # Nonlinear equation
11. def f(Vd):
12.     return Vs - R * Is * (math.exp(Vd / (n * Vt)) - 1) - Vd
13.
14. # Initial guess
15. Vd = 1.0
16. step = 0.001      # 1 mV step
17. stop = 1e-6 # Adequate convergence
18.
19. # Iterative solve
20. for i in range(100000): # max iterations
21.     value = f(Vd)
22.
23.     if abs(value) < stop: #Iterative solving stop condition
24.         break
25.
26.     if value < 0:
27.         Vd -= step      # Vd too high
28.     else:
29.         Vd += step      # Vd too low
30.
31. print(f"Vd ≈ {Vd:.6f} V")
32. print(f"f(Vd) ≈ {f(Vd):.2e}")
33.
```

Figure 10: Iterative solving

Figure 10 uses python to iteratively solve an equation and gives the output in the terminal.



```
Vd ≈ 0.578000 V
f(Vd) ≈ -9.34e-02
```

Figure 11: Python terminal iterative solution

Iteration	0:	Vd = 1.000000 V, f(Vd) = -50539842.594540	(f<0 → Vd too high → DECREASE Vd)
Iteration	1:	Vd = 0.999000 V, f(Vd) = -48632909.205644	(f<0 → Vd too high → DECREASE Vd)
Iteration	2:	Vd = 0.998000 V, f(Vd) = -46797926.865097	(f<0 → Vd too high → DECREASE Vd)
Iteration	3:	Vd = 0.997000 V, f(Vd) = -45032180.767342	(f<0 → Vd too high → DECREASE Vd)
Iteration	4:	Vd = 0.996000 V, f(Vd) = -43333058.539926	(f<0 → Vd too high → DECREASE Vd)

Figure 12: Iteration examples

Figure 12 shows how the iteration was solved and this exactly how spice uses iteration to solve non-linear problems, by converging to a point to find the best fitting solution.

Lab 2

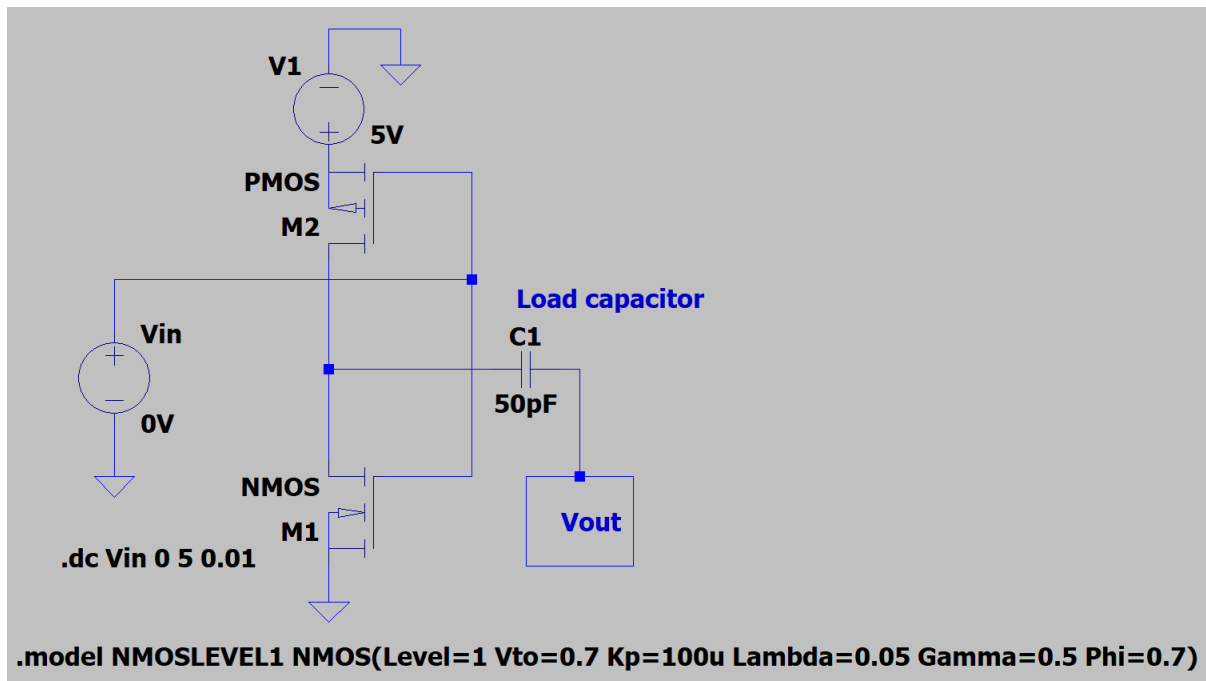


Figure 13: Schematic drawing of CMOS inverter (Task 1A)

```

1. **VDD is connected to VDD and ground and 5 volts DC is provided
2. VDD Vdd 0 DC 5
3. **Vin is connected to ground and in is set at 0V DC initially
4. Vin in 0 DC 0
5. **M1 is the NMOS MOSFET name- drain connected to out and gate connected to in
6. **Source and bulk set to 0/ connected to ground
7. **Level 1 suggests models behaviour
8. **Width = 10u and Length = 1u
9. M1 out in 0 0 NMOSLEVEL1 W=10u L=1u
10. **M2 is the PMOS MOSFET name- drain connected to out and gate connected to in
11. **Source and bulk connected to VDD
12. **Width = 20u and Length = 1u
13. M2 out in Vdd Vdd PMOSLEVEL1 W=20u L=1u
14. **The actual models of the PMOS and NMOS
15. .model NMOSLEVEL1 NMOS (LEVEL=1 VTO=0.7 KP=100u LAMBDA=0.05 GAMMA=0.5 PHI=0.7)
16. .model PMOSLEVEL1 PMOS (LEVEL=1 VTO=-0.7 KP=30u LAMBDA=0.05 GAMMA=0.5 PHI=0.7)
17. **Vin dc sweep from 0 to 5 in 0.01 increments
18. .dc Vin 0 5 0.01
19. **Plot DC vin vs vout
20. .plot DC V(in) V(out)
21. .end

```

Figure 14: Completed netlist for static analysis inverter DC sweep (Task 1B)

Did not include capacitor as not needed for this specific DC analysis.

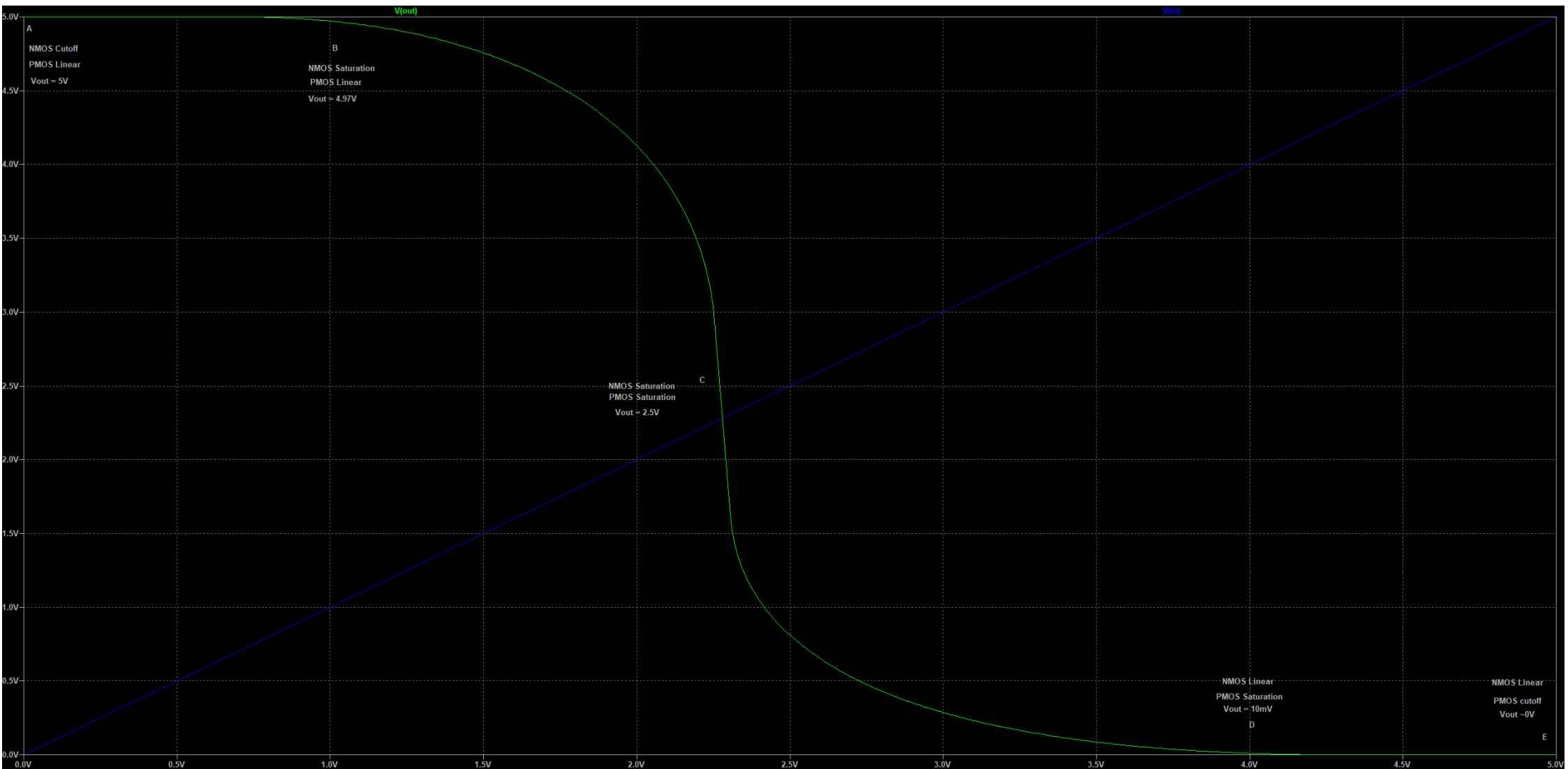


Figure 15: Waveform of V_{out} vs V_{in}

The NMOS pull max voltage is 4.82V. It does not reach 5V because the maximum voltage it can reach is $5V - V_{th}$. When approaching 5V the pull up transistor will stop conducting due to the VGS falling below the threshold frequency.

When $V_{in} = 0$, NMOS will not be conducting as it will be below V_{th} .

If the NMOS inverter drove another CMOS logic gate, The risk is that we would not be power efficient. There is a weak high level of logic therefore slow switching and may have logic errors.

A PMOS will be on at low voltages so will pull up to VDD as will be turned on strongly.

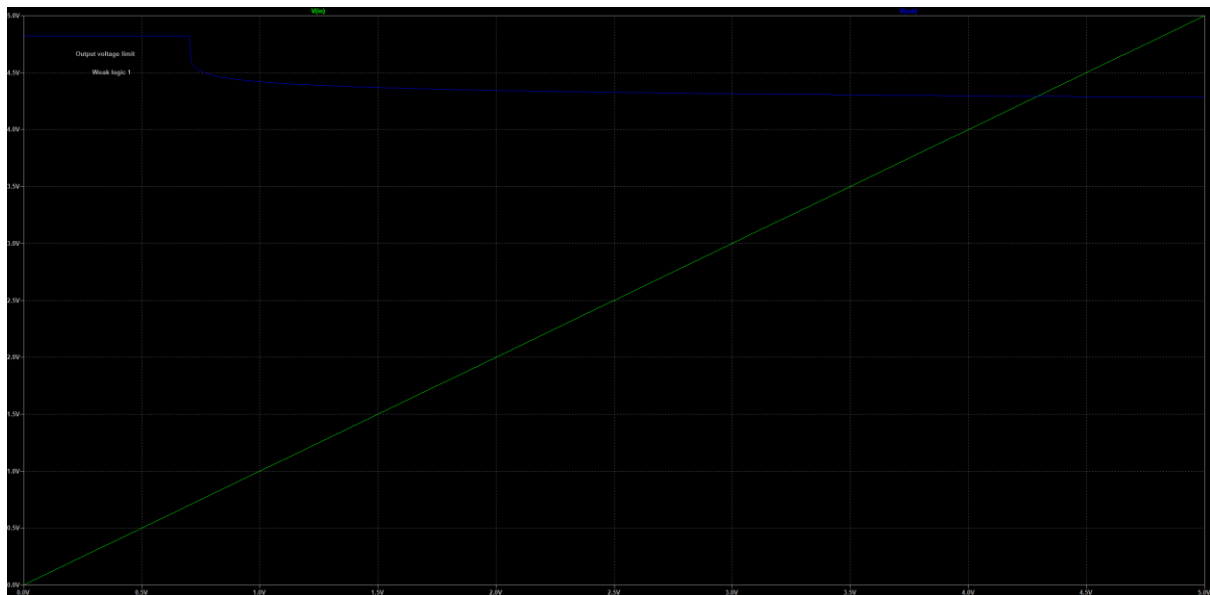


Figure 16: NMOS pull up graph

There is a region at around 600mV and less where a weak logic 1 can be observed. This is because the V_{out} will be equal to $V_{in} - V_{th}$ so cannot produce a 5V signal corresponding to a strong logic 1.

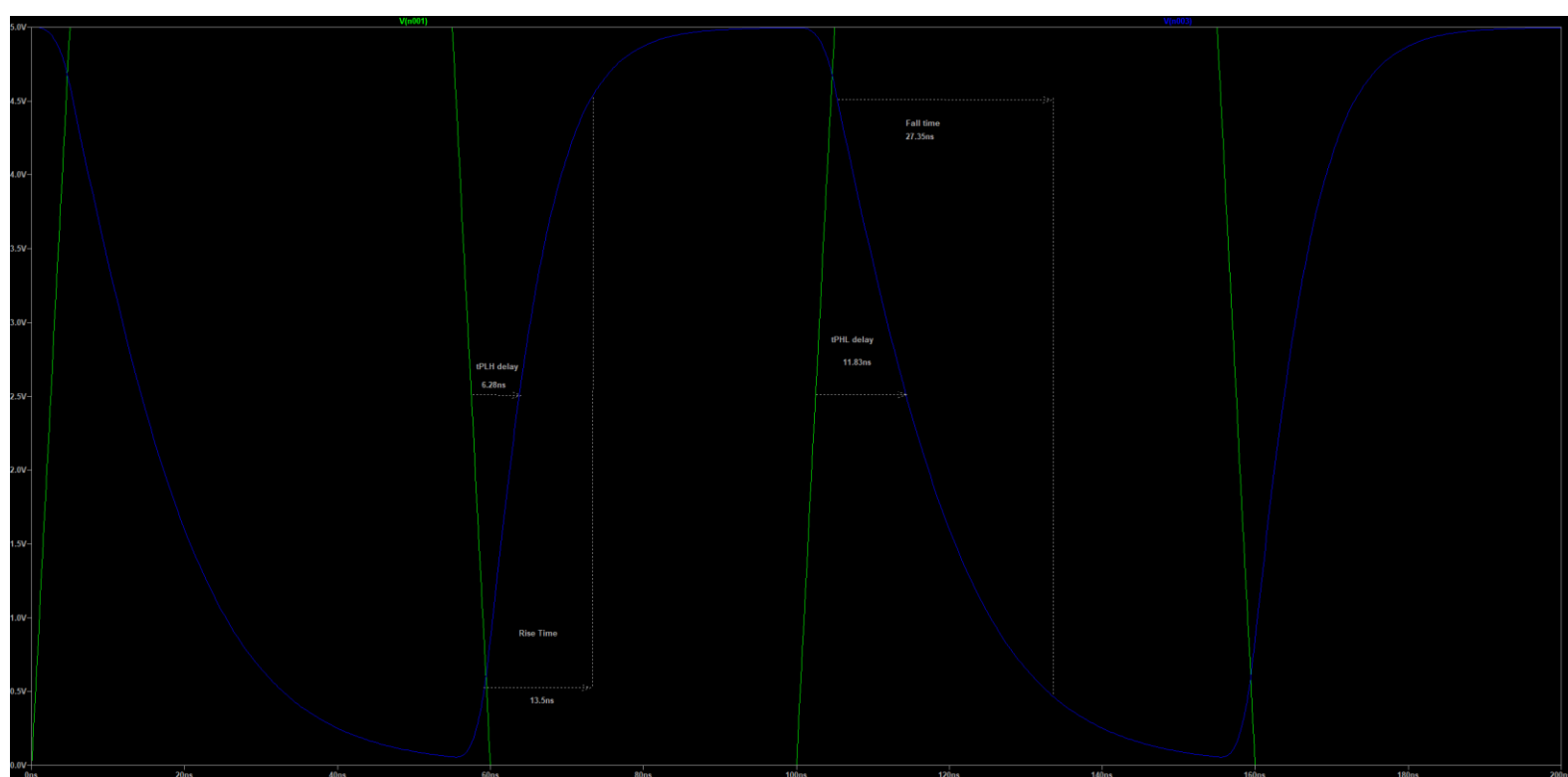


Figure 17: Inverter from specification, green = V_{in} and blue = V_{out}

Rise time = 13.5ns, Fall time = 27.35ns. t_{PLH} = 6.28 ns and t_{PHL} = 11.83ns. The rise time is 10% to 90% and fall time is 90% to 10% of the maximum output. The propagation delays from low to high and high to low measures the response time, since this is an inverter that means that when V_{in} goes low what is the response time that the V_{out} goes high and vice-versa. This is measured at the 50% mark of both signals.

This design does not meet timing specifications because the fall time is not quick enough. This can be changed by changing the NMOS width allowing a faster pull-down.

The resistance can be estimated for both MOSFET with the given specifications:

NMOS: $K_P=200$ $K_P=200\mu$, $V_{th} = 0.4$

PMOS: $K_P=100$ $K_P=100\mu$, $V_{th} = -0.4$

Figure 18: MOSFET specifications

Assuming $V_{DS} = V_{GS} - V_{th} = 2.5$ = NMOS and PMOS in saturation, then figure 19 can be used to estimate resistances. Also assume λ is 0.05.

$$I_{ds} = \frac{200e-6}{2} * \frac{20e-6}{1e-6} * (1 + (0.05 * 2.5)) * (2.9 - 0.4)^2$$

$$I_{ds} = 100e-6 * 20 * 1.125 * 6.25$$

$$I_{ds} = 14.06mA$$

$$R_{nmos} = \frac{2.5}{14e-3} = 178\Omega$$

Replace the equation with PMOS values to get (Remembering VGS is negative and Vth)

$$R_{pmos} = 711\Omega$$

Saturation Region, $v_{ds} \geq v_{gs} - v_{th}$

$$I_{ds} = \frac{KP}{2} \cdot \frac{W_{eff}}{L_{eff}} \cdot (1 + LAMBDA \cdot v_{ds}) \cdot (v_{gs} - v_{th})^2$$

Figure 19: MOSFET equation for saturation

Rise time is 10% to 90% and fall time is 90% to 10%

Charging equation is: $V(t) = VDD * (1 - e^{-\frac{t}{RC}})$

The PMOS is responsible for the rise so therefore:

$$Trise = 0.9VDD - 0.1VDD = -\ln(0.1) RC - -\ln(0.9) RC = 2.2RC = 15.6ns$$

Discharging equations is: $V(t) = VDD * e^{-\frac{t}{RC}}$

NMOS is responsible for fall times therefore:

$$Tfall = 2.2RC = 3.9ns$$

These estimates are different to the simulation because resistance was constant in the estimations and assumed to be in saturation however in the simulation there are multiple modes that the MOSFET's transition from. Therefore, SPICE will use iterative methods to calculate actual resistance values and therefore rise and fall times will be much different.

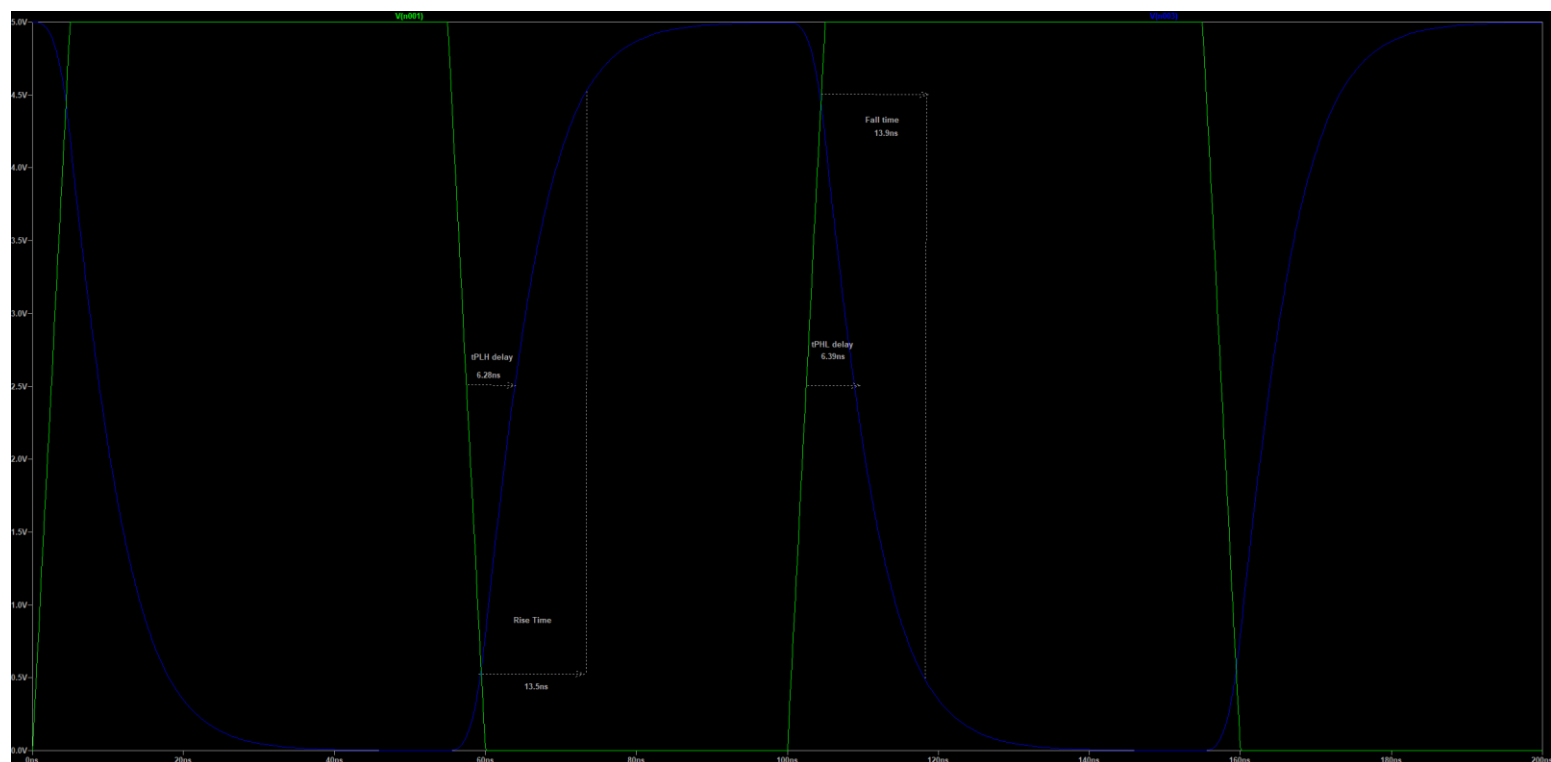


Figure 20: NMOS $W = 20u$

By increasing NMOS width to 20u, the fall time = 13.9ns which falls within specifications and the high low propagation delay fell to 6.39ns. Since only NMOS changes the Rise delay and times stay the same. The issue is that the high value does not seem sustained enough so another design could make sure the logic 1 is stronger.

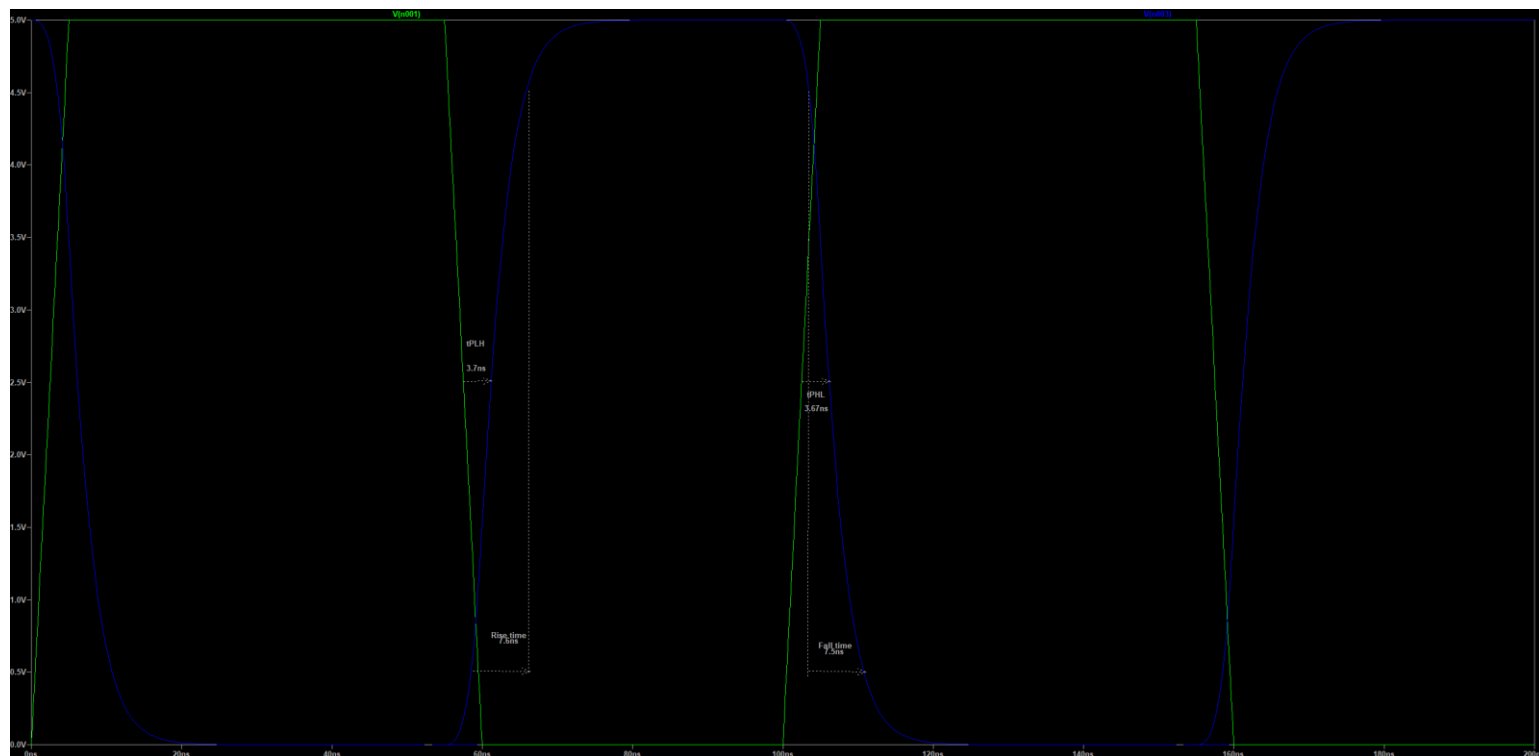


Figure 21: Most Robust design

NMOS has $w = 30\mu$ and $l = 0.5\mu$, PMOS has $W = 20\mu$ and $l = 0.5\mu$. This corresponds to a rise time of 7.6ns and a low high propagation delay of 3.7ns. It also has a fall time of 7.5ns and a high low propagation delay of 3.67ns. This is much faster than the original design for the rise and fall times as well as being more responsive with the input voltage. Figure 21 has the most responsiveness and holds the strongest logic 1 as it maintains 5V for the longest. The sizing of the MOSFET's is not extreme and are easy to satisfy. It satisfies all conditions with roughly similar values allowing symmetry meaning it is robust and versatile for different implementations.

The NMOS produces a faster fall time than a PMOS because electron mobility is greater than hole mobility. This is why PMOS usually are wider than NMOS' to compensate this.

If the widths are halved then using figure 7, beta halves decreasing drain current. This means the delays will increase as the rise and fall times will take longer because the pull up and pull-down power of the MOSFET's will also decrease.

If load capacitance doubles, then rise time and fall time doubles as per equations below figure 19.

The propagation delay symmetry is important as if it doesn't have this then it can cause timing violations as when working with clocks and FPGA design, timing is very important so having symmetry allows designers to understand how to implement it without asynchronous input and outputs.

FPGA

Fixed-point package

```
1. library ieee;
2. use ieee.std_logic_1164.all;
3. use ieee.numeric_std.all;
4.
5. package fp_pkg is
6.   -- Q16.16 fixed-point type
7.   subtype fp32 is signed(31 downto 0);
8.
9.   constant FP_FRAC_BITS : integer := 16;
10.
11.
12.   constant FP_ZERO : fp32 := to_signed(0, 32);
```

```

13. constant FP_ONE : fp32 := to_signed(65536, 32);
14.
15. -- TODO: add any domain-specific constants you need, e.g. R_VALUE
16.
17. -- === Conversions (simulation-friendly) ===
18. function int_to_fp(i : integer) return fp32;
19. function real_to_fp(r : real) return fp32; -- Note: not synthesizable
20. function fp_to_real(x : fp32) return real; -- Note: for testbench/prints
21.
22. -- === Arithmetic (synthesizable) ===
23. function fp_add(a, b : fp32) return fp32; -- with saturation
24. function fp_sub(a, b : fp32) return fp32; -- with saturation
25. function fp_mul(a, b : fp32) return fp32; -- Q16.16 * Q16.16 -> Q16.16
26.
27. end package;
28.
29. package body fp_pkg is
30.
31. function int_to_fp(i : integer) return fp32 is
32. variable result : signed(31 downto 0);
33. begin
34. -- Convert to signed and shift left by 16 bits (equivalent to * 2^16)
35. result := shift_left(to_signed(i, 32), 16);
36. return result;
37. end function;
38.
39. function real_to_fp(r : real) return fp32 is
40. variable result: signed(31 downto 0);
41. variable scaled_real : real;
42. begin
43. -- Scale by 2^16 and convert to integer (rounding)
44. scaled_real := r * 65536.0; --must use this instead of 2 ** 16 to return real value
45. --Removing decimal point
46. return to_signed(integer(scaled_real), 32);
47. end function;
48.
49. function fp_to_real(x : fp32) return real is
50. constant Scale_factor : real := 2.0 ** 16;
51. -- TODO: to_integer(x) / 2^16 as real
52. begin
53. return real(to_integer(x)) / Scale_factor;
54. end function;
55.
56. -- fp_add with sign-based overflow detection
57. function fp_add(a, b : fp32) return fp32 is
58. variable a33 : signed(32 downto 0) := resize(a, 33);
59. variable b33 : signed(32 downto 0) := resize(b, 33);
60. variable sum33 : signed(32 downto 0);
61. constant MAX_VAL : fp32 := to_signed(2147483647, 32);
62. constant MIN_VAL : fp32 := to_signed(-2147483648, 32);
63. begin
64. sum33 := a33 + b33;
65.
66.
67. if (a(31) = b(31)) and (sum33(32) /= sum33(31)) then
68. if a(31) = '0' then
69. return MAX_VAL; -- positive saturation
70. else
71. return MIN_VAL; -- negative saturation
72. end if;

```

```

73.     else
74.         return fp32(sum33(31 downto 0));
75.     end if;
76. end function;
77.
78. function fp_sub(a, b : fp32) return fp32 is
79.     variable a33 : signed(32 downto 0) := resize(a, 33);
80.     variable b33 : signed(32 downto 0) := resize(b, 33);
81.     variable sub33 : signed(32 downto 0);
82.     constant MAX_VAL : fp32 := to_signed(2147483647, 32);
83.     constant MIN_VAL : fp32 := to_signed(-2147483648, 32);
84. begin
85.     sub33 := a33 - b33;
86.
87.     --Check if the 33-bit difference overflows
88.     if (a(31) /= b(31)) and (sub33(32) /= sub33(31)) then
89.         if a(31) = '0' then
90.             return MAX_VAL; -- positive saturation
91.         else
92.             return MIN_VAL; -- negative saturation
93.         end if;
94.     else
95.         return fp32(sub33(31 downto 0));
96.     end if;
97. end function;
98.
99. function fp_mul(a, b : fp32) return fp32 is
100.     variable product : signed(63 downto 0);
101.     variable result : signed(31 downto 0);
102.     constant MAX_VAL : fp32 := to_signed(2147483647, 32);
103.     constant MIN_VAL : fp32 := to_signed(-2147483648, 32);
104. begin
105.     -- 64-bit multiply
106.     product := signed(a) * signed(b);
107.
108.     -- Extract Q16.16 (top-shifted)
109.     result := product(47 downto 16);
110.     if (product(63 downto 48) /= x"0000") and (product(63 downto 48) /= x"FFFF") then
111.         if product(63) = '0' then
112.             return MAX_VAL;
113.         else
114.             return MIN_VAL;
115.         end if;
116.     end if;
117.
118.     return fp32(result);
119. end function;
120.
121.
122. end package body;

```

IPSP

Single-Cycle

```

1. library ieee;
2. use ieee.std_logic_1164.all;
3. use ieee.numeric_std.all;

```



```

4. use work.fp_pkg.all;
5.
6. entity ipsp_single_cycle is
7.   port (
8.     clk      : in  std_logic;
9.     rst      : in  std_logic;
10.    en       : in  std_logic;
11.
12.    valid_in  : in  std_logic;
13.    a_in      : in  fp32; -- coefficient
14.    x_in      : in  fp32; -- input data
15.    y_in      : in  fp32; -- previous partial sum
16.
17.    valid_out : out std_logic;
18.    a_out     : out fp32; -- pass-through (aligned)
19.    x_out     : out fp32; -- pass-through (aligned)
20.    y_out     : out fp32; -- new partial sum
21.  );
22. end entity;
23.
24. architecture rtl of ipsp_single_cycle is
25.   signal valid_toggle : std_logic := '0';
26.   signal a_reg, x_reg, y_reg : fp32 := (others => '0');
27.   signal valid_in_reg : std_logic := '0';
28. begin
29.   -- Main clocked process
30.   process(clk)
31.     variable product: fp32;
32.     variable sum: fp32;
33.   begin
34.     if rising_edge(clk) then
35.       if rst = '1' then
36.         valid_toggle <= '0';
37.         valid_out <= '0';
38.         a_out <= (others => '0');
39.         x_out <= (others => '0');
40.         y_out <= (others => '0');
41.         a_reg <= (others => '0');
42.         x_reg <= (others => '0');
43.         y_reg <= (others => '0');
44.         valid_in_reg <= '0';
45.       elsif en = '1' then
46.         -- Register inputs
47.         a_reg <= a_in;
48.         x_reg <= x_in;
49.         y_reg <= y_in;
50.         valid_in_reg <= valid_in;
51.
52.         -- Process registered values
53.         if valid_in_reg = '1' then
54.           product := fp_mul(a_reg, x_reg);
55.           sum := fp_add(product, y_reg);
56.           y_out <= sum;
57.
58.           -- Toggle and output valid signal
59.           valid_toggle <= not valid_toggle;
60.           valid_out <= valid_toggle; -- Output before toggling
61.         else
62.           y_out <= y_reg;
63.           valid_out <= '0';

```

```

64.         end if;
65.
66.         -- Pass through
67.         a_out <= a_reg;
68.         x_out <= x_reg;
69.     end if;
70. end if;
71. end process;
72. end architecture;

```

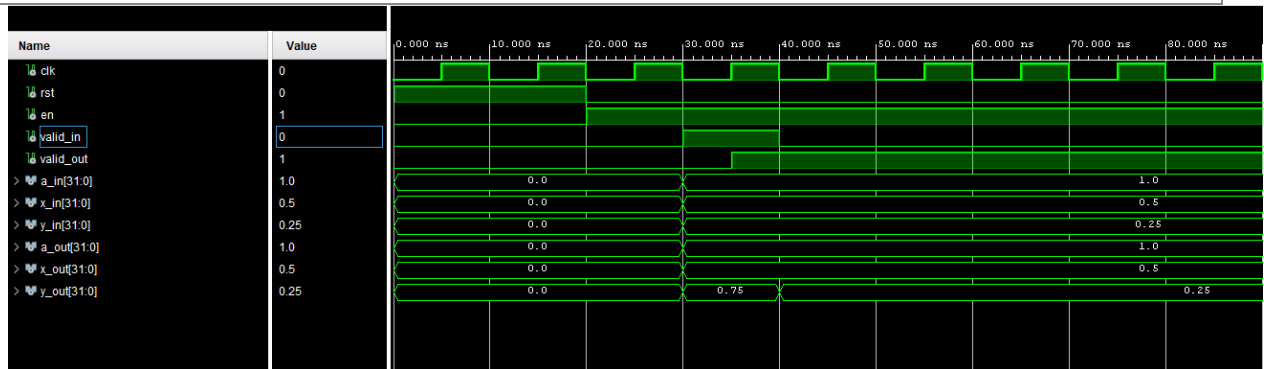


Figure 22: Waveform of the single-cycle IPSP

The clock cycle is 10ns, Figure 22 shows that it completes in one clock cycle after valid in is 1 as y_out equals 0.75 $[(a \times x) + y]$ when valid in = 1. Reset equals 1 sets enable to 0. In the testbench it waits 10ns after enable is set to 1. Then Valid in and valid out are within the same clock cycle therefore the y_out can be processed.

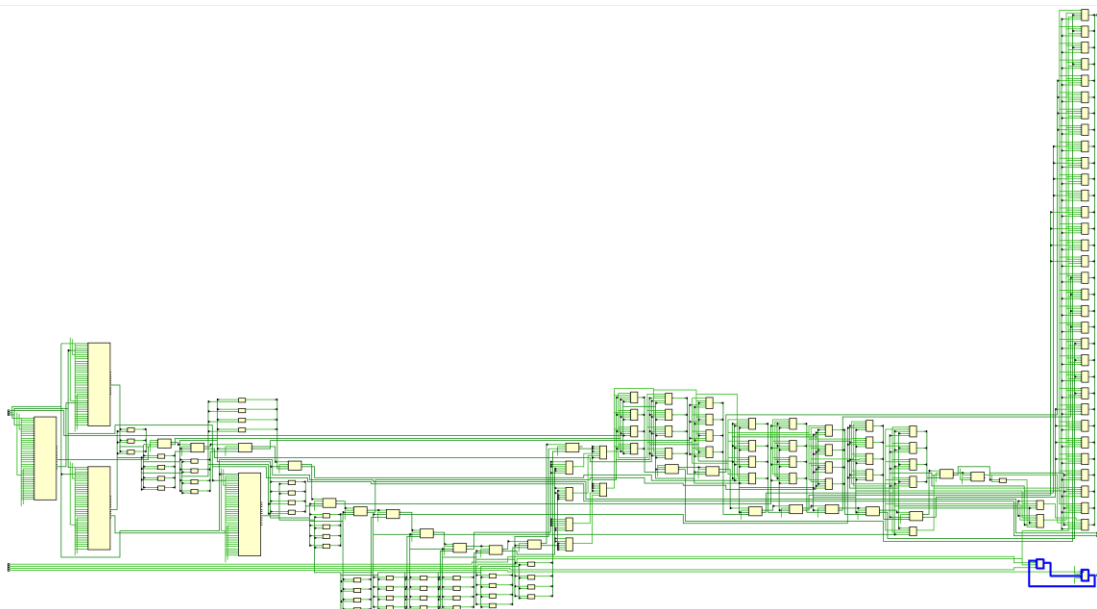


Figure 23: Synthesised schematic with critical path delay of single-cycle IPSP

Design Timing Summary					
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS): 18.830 ns		Worst Hold Slack (WHS): 0.246 ns		Worst Pulse Width Slack (WPWS): 9.500 ns	
Total Negative Slack (TNS): 0.000 ns		Total Hold Slack (THS): 0.000 ns		Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 0		Number of Failing Endpoints: 0		Number of Failing Endpoints: 0	
Total Number of Endpoints: 1		Total Number of Endpoints: 1		Total Number of Endpoints: 1	
All user specified timing constraints are met.					

Figure 24: Timing report on synthesised design single-cycle IPSP

Adding the clock constraint at 20 ns means 1/20ns = 50MHz which is the minimum frequency the design can operate at.

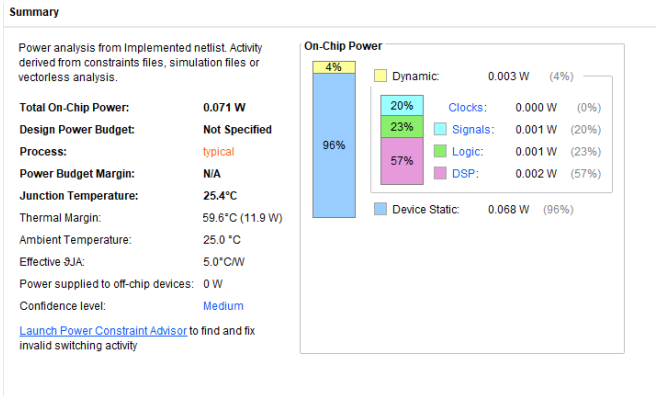


Figure 25: Power summary

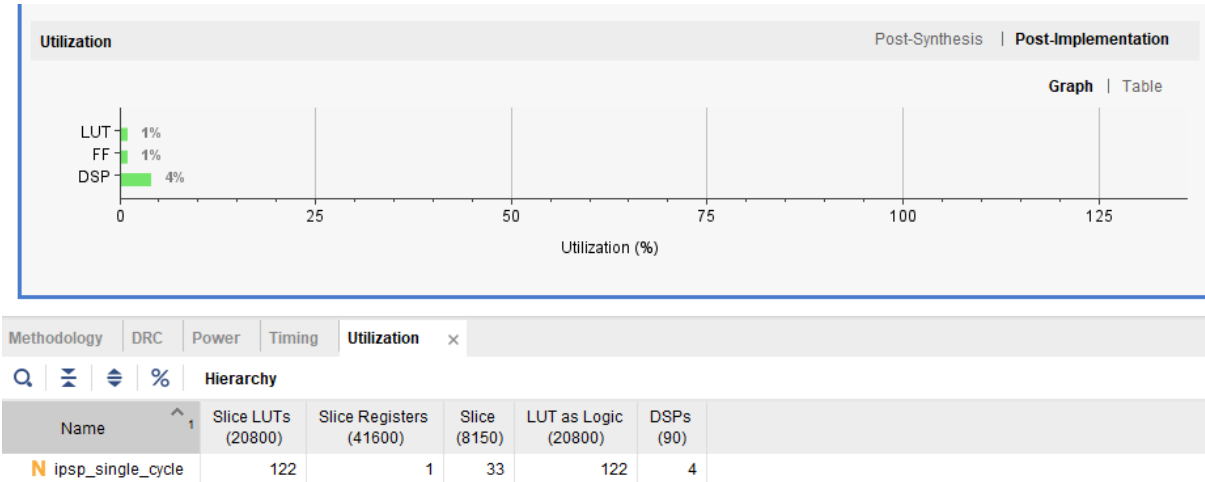


Figure 26: Utilisation

Figure 26 shows the key blocks used in the FPGA.

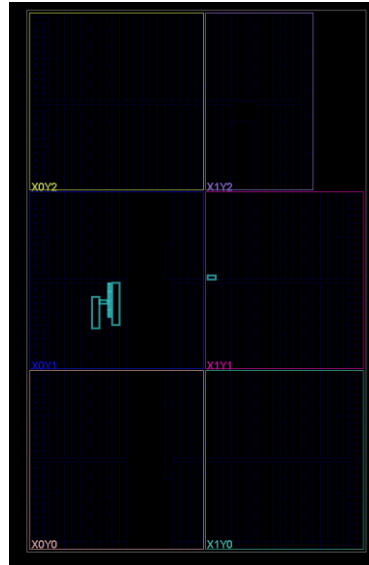


Figure 27: Implemented design single-cycle IPSP

Figure 27 shows how the hardware would look spatially with the design created.

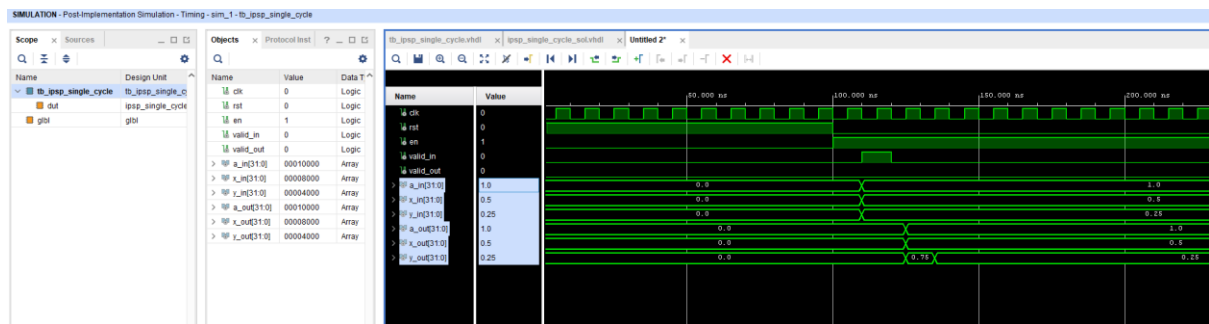


Figure 28: Post-implementation timing

Pipelined IPSP

```

1. library ieee;
2. use ieee.std_logic_1164.all;
3. use ieee.numeric_std.all;
4. use work.fp_pkg.all;
5.
6. entity ipsp_pipelined is
7.   port (
8.     clk      : in  std_logic;
9.     rst      : in  std_logic;
10.    en       : in  std_logic;
11.
12.    valid_in  : in  std_logic;
13.    a_in      : in  fp32;
14.    x_in      : in  fp32;
15.    y_in      : in  fp32;
16.
17.    valid_out : out std_logic;
18.    a_out     : out fp32;
19.    x_out     : out fp32;
20.    y_out     : out fp32;
21.  );
22. end entity;

```

```

23.
24. architecture rtl of ipsp_pipelined is
25.     -- Stage registers (declare and comment each stage)
26.     -- S1: input regs
27.     signal s1_valid : std_logic; --Valid signal for stage 1
28.     signal s1_a, s1_x, s1_y : fp32;--Inputs for stage 1
29.
30.     -- S2: multiply
31.     signal s2_valid : std_logic; --Valid signal for stage 2
32.     signal s2_a, s2_x : fp32; --Passthrough
33.     signal s2_prod : fp32;--Stage 2 multiplication result
34.
35.     -- S3: register product
36.     signal s3_valid : std_logic; --Valid signal for stage 3
37.     signal s3_a, s3_x : fp32; --Passthrough
38.     signal s3_prod, s3_y : fp32; --Registers for the product and delayed y
39.
40.     -- S4: add and output
41.     signal s4_valid : std_logic; --Valid signal for stage 4
42.     signal s4_a, s4_x, s4_y : fp32; --Final output registers
43. begin
44.
45.     process(clk)
46.     begin
47.         if rising_edge(clk) then
48.             if rst = '1' then
49.                 -- TODO: reset all stage registers
50.                 s1_valid <= '0'; --Setting all registers to 0 state.
51.                 s2_valid <= '0';
52.                 s3_valid <= '0';
53.                 s4_valid <= '0';
54.                 s1_a <= (others => '0');
55.                 s1_x <= (others => '0');
56.                 s1_y <= (others => '0');
57.                 s2_a <= (others => '0');
58.                 s2_x <= (others => '0');
59.                 s2_prod <= (others => '0');
60.                 s3_a <= (others => '0');
61.                 s3_x <= (others => '0');
62.                 s3_prod <= (others => '0');
63.                 s3_y <= (others => '0');
64.                 s4_a <= (others => '0');
65.                 s4_x <= (others => '0');
66.                 s4_y <= (others => '0');
67.
68.             elsif en = '1' then
69.                 -- === Stage 1: register inputs ===
70.                 s1_valid <= valid_in; --Registering all inputs to another register allowing
synchronous propagation
71.                 s1_a <= a_in;
72.                 s1_x <= x_in;
73.                 s1_y <= y_in;
74.
75.                 -- === Stage 2: multiply ===
76.                 s2_valid <= s1_valid; --Valid state propagates
77.                 s2_a <= s1_a; --Passthrough a
78.                 s2_x <= s1_x; --Passthrough x
79.                 s2_prod <= fp_mul(s1_a, s1_x); --Multiplication using the fp package
80.
81.                 -- === Stage 3: register product (+ align y) ===

```

```

82.      s3_valid <= s2_valid; --Valid state propagates
83.      s3_a <= s2_a; --Passthrough a
84.      s3_x <= s2_x; --Passthrough x
85.      s3_prod <= s2_prod; --Registering the product to stage 3
86.      -- Delay y to align with the product for addition
87.      s3_y <= s1_y; --Registering y to stage 3 to allow timing alignment
88.
89.      -- === Stage 4: add and drive outputs ===
90.      s4_valid <= s3_valid; --Valid states propagates
91.      s4_a <= s3_a; --Passthrough a
92.      s4_x <= s3_x; --Passthrough B
93.      s4_y <= fp_add(s3_prod, s3_y); --Product + y
94.  end if;
95.  end if;
96. end process;
97.
98. -- Outputs
99. valid_out <= s4_valid; --Outputs are then connected to the outputs
100. a_out    <= s4_a;
101. x_out    <= s4_x;
102. y_out    <= s4_y;
103.
104. end architecture;

```

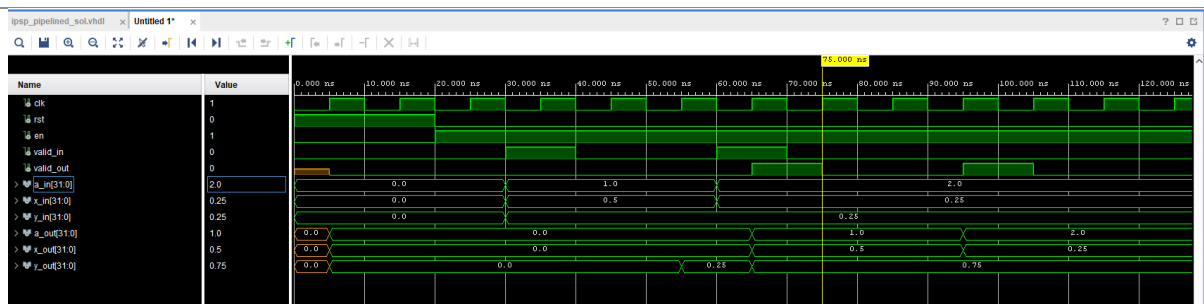


Figure 29: IPSP pipeline waveforms

Initially in figure 29, the orange waveform is to initially set all registers to 0 to clear them just in case they had values in from a previous test and this occurs during reset. When enable is active (and waiting 10ns for testbench) the inputs are then validated in to registers. Then after IPSP arithmetic, and y being passed through to the final stage for timing alignment, y_out gets the calculated result. Two stimuli are used to check that it is working correctly using different numbers to get the same result ($a = 1, 2$ and $x = 0.5, 0.25$).

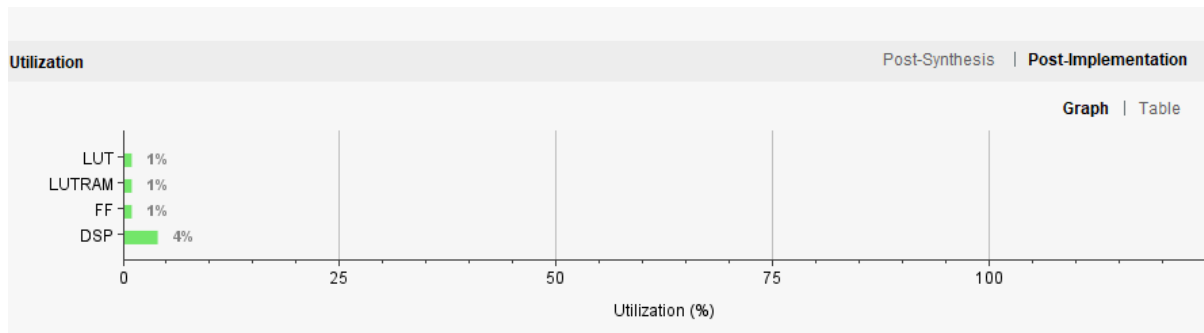


Figure 30: Utilisation

Figure 30 shows that pipelining uses LUTRAM unlike single cycle because it requires registers to hold values between intermediate stages.

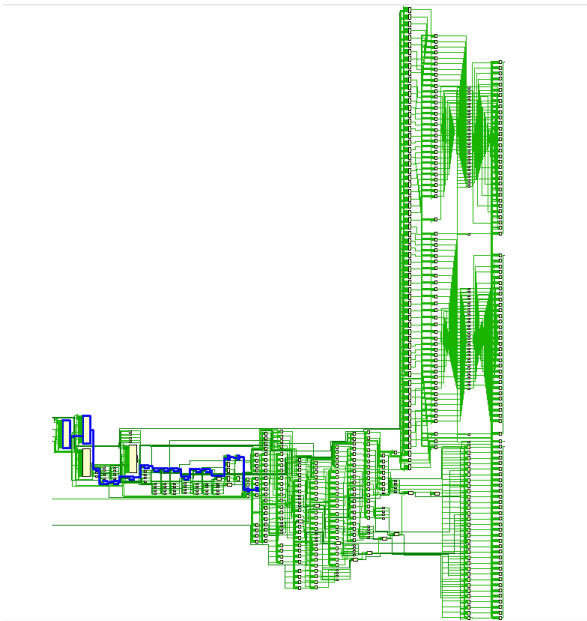


Figure 31: Synthesised schematic with critical path delay of pipelined IPSP

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 7.361 ns	Worst Hold Slack (WHS): 0.163 ns	Worst Pulse Width Slack (WPWS): 9.020 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 260	Total Number of Endpoints: 260	Total Number of Endpoints: 358
All user specified timing constraints are met.		

Figure 32: Timing report of pipeline IPSP

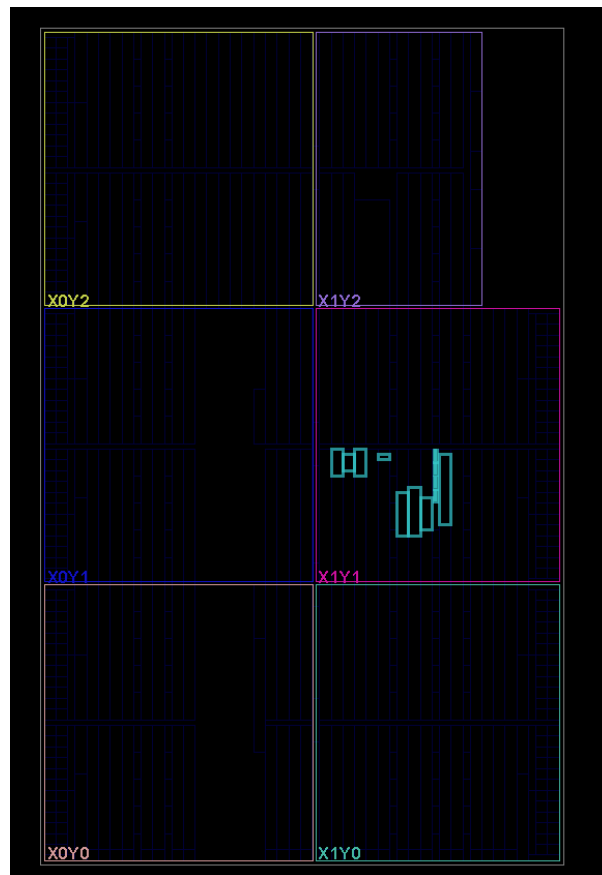


Figure 33: Implemented design pipeline IPSP

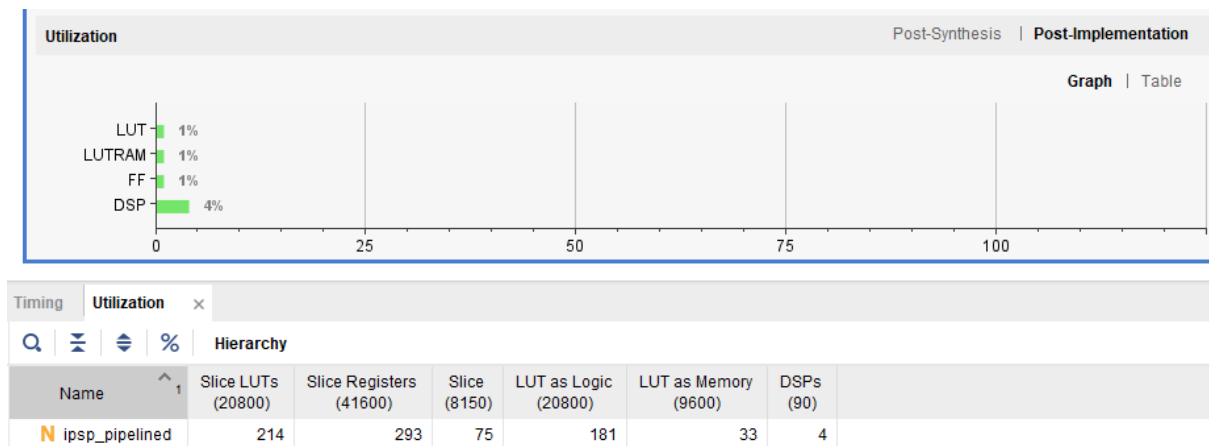


Figure 34: utilisation

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: 0.074 W
Design Power Budget: Not Specified
Process: typical
Power Budget Margin: N/A
Junction Temperature: 25.4°C
Thermal Margin: 59.6°C (11.9 W)
Ambient Temperature: 25.0 °C
Effective θ_{JA} : 5.0°C/W
Power supplied to off-chip devices: 0 W
Confidence level: Medium

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

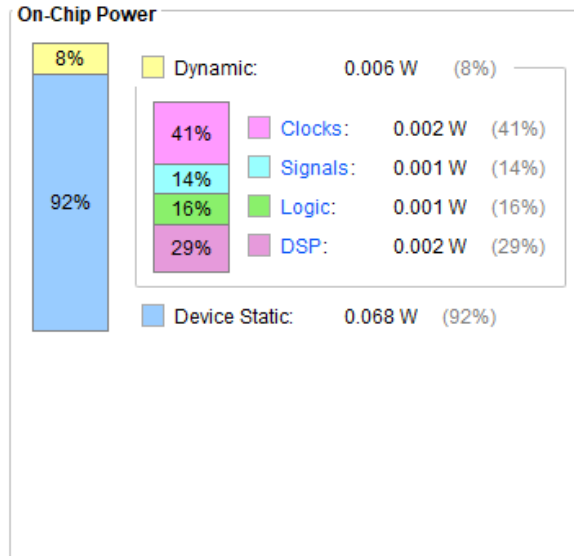


Figure 35: Power utilisation IPSP pipeline

Looking at the timing reports of both the different designs, the conclusion of critical path delay can be made. For the pipeline design, $20 - 7.361 = 12.639\text{ns}$ critical path delay = $1/12.639\text{ns} = 79\text{ MHz}$ max frequency. Changing the clock constraint to 11ns (90MHz) clock period led the pipelined design to fail however at 12.693ns (79MHz) it worked.

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): -0.039 ns	Worst Hold Slack (WHS): 0.163 ns	Worst Pulse Width Slack (WPWS): 5.320 ns
Total Negative Slack (TNS): -0.076 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 2	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 260	Total Number of Endpoints: 260	Total Number of Endpoints: 358

Timing constraints are not met.

Figure 36: Pipeline failing at new time constraint (11ns (90MHz))

The same can be said for the single cycle IPSP, $20 - 18.83 = 1.17\text{ns}$ critical path delay = $1/1.17\text{ns} = 855\text{MHz}$. The same test was done for single cycle by changing the clock to 17ns and having a timing failure but above 18.83ns led to a success.

Although the single-cycle can operate on a higher maximum frequency clock, the pipelining is still essential and performs better as once the IPSP is fully operational in the pipeline, the cycles will all work in parallel therefore processing more operations than the single-cycle.

The multiplication is mapped to the FPGA primitive dsp48 as seen in figure 37.

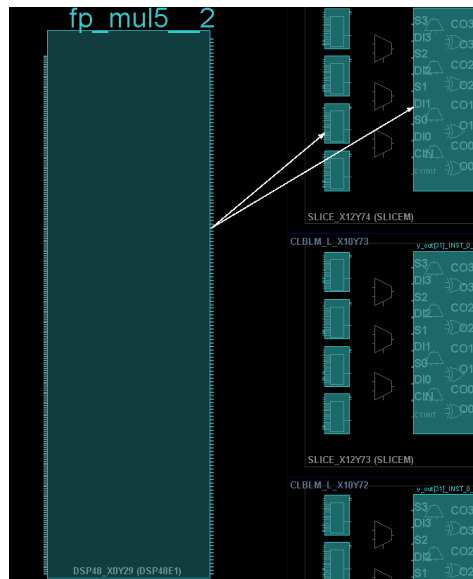


Figure 37: FPGA primitives

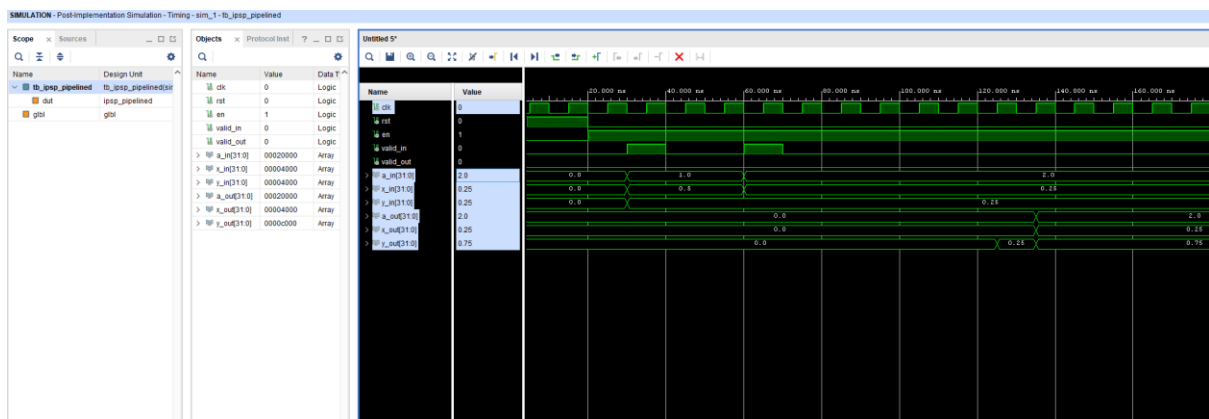


Figure 38: Post-implementation timing simulation

DIVSUB

Single-Cycle DIVSUB

```

1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3. use IEEE.NUMERIC_STD.ALL;
4.
5. use work.fp_pkg.all;
6.
7. entity divsub is
8.   port (
9.     clk      : in  std_logic;
10.    rst       : in  std_logic;
11.    a_in      : in  fp32;           -- Q16.16
12.    b_in      : in  fp32;           -- Q16.16
13.    y_in      : in  fp32;           -- Q16.16
14.    valid_in  : in  std_logic;
15.    x_out     : out fp32;           -- Q16.16 result
16.    valid_out : out std_logic
17.  );
18. end entity;
```

```

19.
20. architecture rtl of divsub is
21.
22.     function fp_divide(a : fp32; b : fp32) return fp32 is
23.         variable num_s : signed(47 downto 0); -- 32-bit + 16-bit preparation for division
24.         variable den_s : signed(31 downto 0); -- 32-bit denominator
25.         variable quo_s : signed(31 downto 0); -- 32-bit quotient
26.     begin
27.         den_s := signed(b);
28.         if den_s = 0 then
29.             return (others => '0'); -- Policy: return 0 on divide-by-zero (explicit constant from
package)
30.         else
31.             -- Scale numerator by 2^16 to keep Q16.16 after integer division
32.             num_s := resize(signed(a), 48) sll 16;
33.             quo_s := resize(num_s / den_s, 32);
34.             return fp32(quo_s);
35.         end if;
36.     end function;
37.
38.     signal x_r      : fp32 := (others => '0');
39.     signal v_r      : std_logic := '0';
40.
41.     signal diff_q    : fp32;
42.
43. begin
44.
45.     --subtraction: b - y
46.     diff_q <= fp_sub(b_in, y_in);
47.
48.     -- Register outputs for a clean 1-cycle latency
49.     process(clk)
50.     begin
51.         if rising_edge(clk) then
52.             if rst = '1' then
53.                 x_r <= (others => '0');
54.                 v_r <= '0';
55.             else
56.                 -- Division via local helper (keeps everything in fp_data)
57.                 x_r <= fp_divide(a_in, diff_q);
58.                 v_r <= valid_in;
59.             end if;
60.         end if;
61.     end process;
62.
63.     x_out      <= x_r;
64.     valid_out <= v_r;
65.
66. end architecture;

```

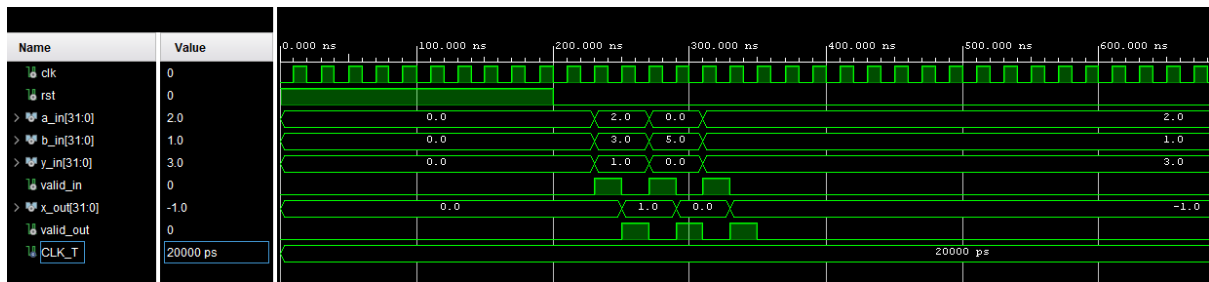


Figure 39: Waveform of DIVSUB

This DIVSUB completes in one cycle as valid_in and valid_out are alternating therefore the x_out is instantly validated. There are clear waveforms for the testbench values, and the CLK_T is the clock constraint. Again, reset starts as 1 to reset the system before going to 0 and since it is single cycle, enable is not needed as the process does not need freezing.

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): -129.414 ns	Worst Hold Slack (WHS): 1.657 ns	Worst Pulse Width Slack (WPWS): 9.500 ns
Total Negative Slack (TNS): -4114.037 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 32	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 65	Total Number of Endpoints: 65	Total Number of Endpoints: 33

Timing constraints are not met.

Figure 40: Timing report

Timing violation does not allow post-implementation timing simulation.

Multi-cycle DIVSUB

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4  use work.fp_pkg.all;
5
6
7  entity divsub is
8  port (
9      clk      : in  std_logic;
10     rst      : in  std_logic;
11     enable   : in  std_logic;
12     a_in     : in  fp32;
13     b_in     : in  fp32;
14     y_in     : in  fp32;
15     x_out    : out fp32;
16     valid_in : in  std_logic;
17     valid_out : out std_logic
18 );
19 end entity divsub;
20
21 architecture simple of divsub is
22
23     -----
24     -- Simple FIFO to decouple input from divider
25     -----
26
27     type fifo_data_t is record
28         a : fp32;
29         b : fp32;
30         y : fp32;
31     end record;
32
33     constant FIFO_DEPTH : integer := 32;
34     type fifo_array_t is array (0 to FIFO_DEPTH-1) of fifo_data_t;
35     signal fifo : fifo_array_t;
36     signal wr_ptr : integer range 0 to FIFO_DEPTH-1 := 0;
37     signal rd_ptr : integer range 0 to FIFO_DEPTH-1 := 0;
38     signal count : integer range 0 to FIFO_DEPTH := 0;
39
40     -----
41     -- Control FSM
42     -----
43     type state_t is (IDLE, PROCESS_DATA, DIVIDING, OUTPUT_RESULT);
44     signal state : state_t := IDLE;
45
46     -----
47     -- Latched work registers
48     -----
49     signal work_a, work_b, work_y : fp32 := FP_ZERO;
50     signal div_sign : std_logic := '0';
51
52     -----
53     -- Divider internals (unsigned magnitude division; sign applied at the end)
54     -- We build a 48-bit dividend = |b-y| << 16 to preserve Q16.16 fractional bits
55     -----
56     signal dividend_reg : unsigned(47 downto 0) := (others => '0'); -- 48-bit dividend
57     signal divisor_reg : unsigned(31 downto 0) := (others => '0'); -- 32-bit divisor magnitude
58     signal divisor_ext : unsigned(32 downto 0) := (others => '0'); -- 33-bit for compare/sub
59     signal quotient_reg : unsigned(31 downto 0) := (others => '0'); -- 32-bit Q16.16 quotient magnitude
60     signal remainder_reg : unsigned(32 downto 0) := (others => '0'); -- 33-bit remainder (guard bit)
61     signal div_counter : integer range 0 to 48 := 0;
62     signal div_active : std_logic := '0';
63     signal top_q_nonzero : std_logic := '0';
64
65     -----
66     -- Output regs and constants
67     -----
68     signal result : fp32 := FP_ZERO;
69     signal result_valid : std_logic := '0';
70
71     constant POS_MAX : unsigned(31 downto 0) := x"7FFF_FFFF";
72     constant NEG_MAX : unsigned(31 downto 0) := x"8000_0000";

```

```

73 begin
74
75 main_process : process(clk)
76 -- Variables for arithmetic inside the clocked process
77 variable diff_val      : signed(31 downto 0);
78 variable a_val         : signed(31 downto 0);
79 variable diff_abs      : unsigned(31 downto 0);
80 variable a_abs         : unsigned(31 downto 0);
81 variable temp_remainder : unsigned(32 downto 0);
82 variable new_remainder : unsigned(32 downto 0);
83 variable qbit          : std_logic;
84
85 -- FIFO control
86 variable wr_en : boolean;
87 variable rd_en : boolean;
88
89 -- Post-division pack
90 variable q_mag_sat : unsigned(31 downto 0);
91 variable q_signed  : signed(31 downto 0);
92 begin
93 if rising_edge(clk) then
94     if rst = '1' then
95         -- Reset FIFO
96         wr_ptr <= 0;
97         rd_ptr <= 0;
98         count <= 0;
99         -- Reset FSM and working regs
100         state <= IDLE;
101         work_a <= FP_ZERO;
102         work_b <= FP_ZERO;
103         work_y <= FP_ZERO;
104         div_sign <= '0';
105         -- Reset divider
106         dividend_reg <= (others => '0');
107         divisor_reg <= (others => '0');
108         divisor_ext <= (others => '0');
109         quotient_reg <= (others => '0');
110         remainder_reg <= (others => '0');
111         div_counter <= 0;
112         div_active <= '0';
113         top_q_nonzero <= '0';
114         -- Reset outputs
115         result <= FP_ZERO;
116         result_valid <= '0';
117
118     else
119         -- Default deassert valid
120         result_valid <= '0';
121
122         -----
123         -- FIFO write path
124         -----
125         wr_en := (enable = '1') and (valid_in = '1') and (count < FIFO_DEPTH);
126         if wr_en then
127             fifo(wr_ptr).a <= a_in;
128             fifo(wr_ptr).b <= b_in;
129             fifo(wr_ptr).y <= y_in;
130             if wr_ptr = FIFO_DEPTH-1 then
131                 wr_ptr <= 0;
132             else
133                 wr_ptr <= wr_ptr + 1;
134             end if;
135         end if;
136

```

```

137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208

```

```

-----
-- FIFO read path (only when idle and divider not busy)
-----

rd_en := (state = IDLE) and (count > 0) and (div_active = '0');
if rd_en then
    work_a <= fifo(rd_ptr).a;
    work_b <= fifo(rd_ptr).b;
    work_y <= fifo(rd_ptr).y;
    if rd_ptr = FIFO_DEPTH-1 then
        rd_ptr <= 0;
    else
        rd_ptr <= rd_ptr + 1;
    end if;
end if;

-- FIFO count update (atomic)
if wr_en and (not rd_en) then
    count <= count + 1;
elsif rd_en and (not wr_en) then
    count <= count - 1;
else
    count <= count; -- both or neither → unchanged
end if;

-----
-- FSM
-----

case state is

    when IDLE =>
        if rd_en then
            state <= PROCESS_DATA;
        else
            state <= IDLE;
        end if;

    when PROCESS_DATA =>
        -- Compute signed difference (b - y)
        diff_val := signed(work_b) - signed(work_y);
        a_val    := signed(work_a);

        -- Early divide-by-zero handling
        if a_val = 0 then
            result      <= FP_ZERO; -- define as 0 (or choose a policy)
            result_valid <= '1';
            state       <= IDLE;
        else
            -- Determine sign of final result
            div_sign <= diff_val(31) xor a_val(31);

            -- Absolute magnitudes
            if diff_val(31) = '1' then
                diff_abs := unsigned(-diff_val);
            else
                diff_abs := unsigned(diff_val);
            end if;

            if a_val(31) = '1' then
                a_abs := unsigned(-a_val);
            else
                a_abs := unsigned(a_val);
            end if;

            -- Build 48-bit dividend = |b-y| << 16 (append 16 zero LSBs)
            dividend_reg <= diff_abs & to_unsigned(0, 16);

            -- Load divisor and its 33-bit extension
            divisor_reg  <= a_abs;
            divisor_ext  <= '0' & a_abs;

```

```

209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259

-- Reset divider machine
quotient_reg <= (others => '0');
remainder_reg <= (others => '0');
div_counter <= 48; -- process all 48 dividend bits
div_active <= '1';
top_q_nonzero <= '0';

state <= DIVIDING;
end if;

-----
when DIVIDING =>
if div_counter > 0 then
-- Shift remainder left by 1 and bring in the next dividend MSB
temp_remainder := remainder_reg(31 downto 0) & dividend_reg(47);

-- Trial subtraction
if temp_remainder >= divisor_ext then
new_remainder := temp_remainder - divisor_ext;
qbit := '1';
else
new_remainder := temp_remainder;
qbit := '0';
end if;

-- Commit new remainder
remainder_reg <= new_remainder;

-- We run 48 cycles: first 16 produce the top quotient bits we ignore
-- for output (but track to detect positive overflow), last 32 fill quotient_reg
if div_counter <= 32 then
quotient_reg <= quotient_reg(30 downto 0) & qbit; -- keep lower 32 bits (Q16.16)
else
if qbit = '1' then
top_q_nonzero <= '1'; -- remember integer overflow beyond 16 bits
end if;
quotient_reg <= quotient_reg; -- unchanged
end if;

-- Shift dividend to expose next bit
dividend_reg <= dividend_reg(46 downto 0) & '0';

-- Next iteration
div_counter <= div_counter - 1;
else
-- All bits consumed
div_active <= '0';
state <= OUTPUT_RESULT;
end if;
end process;

-----
when OUTPUT_RESULT =>
-- Saturate magnitude according to sign
if div_sign = '0' then
if (top_q_nonzero = '1') or (quotient_reg > POS_MAX) then
q_mag_sat := POS_MAX; -- positive overflow -> clamp
else
q_mag_sat := quotient_reg; -- within range
end if;
else
if quotient_reg > NEG_MAX then
q_mag_sat := NEG_MAX; -- negative magnitude too large -> clamp to -32768.0
else
q_mag_sat := quotient_reg;
end if;
end if;

-- Apply sign over full 32-bit width
if div_sign = '1' then
q_signed := -signed(std_logic_vector(q_mag_sat));
else
q_signed := signed(std_logic_vector(q_mag_sat));
end if;

-- Register outputs
result <= q_signed;
result_valid <= '1';
state <= IDLE;

end case; -- state

end if; -- rst
end if;
end process;

-- Output ports
x_out <= result;
valid_out <= result_valid;
end architecture simple;

```

Comments are too long to format properly.

The dividend register will hold the value of the numerator (b-y). It has size 48 bits because it holds a 32-bit number being shifted to the left by 16 bits to prepare for division and preserve bits. The Divisor register is 32-bit size and will hold the denominator value (a). There is an extended version that is 33-bits in size for easier subtraction comparisons. The remainder register is 33-bit size to store partial remainders during the division process.

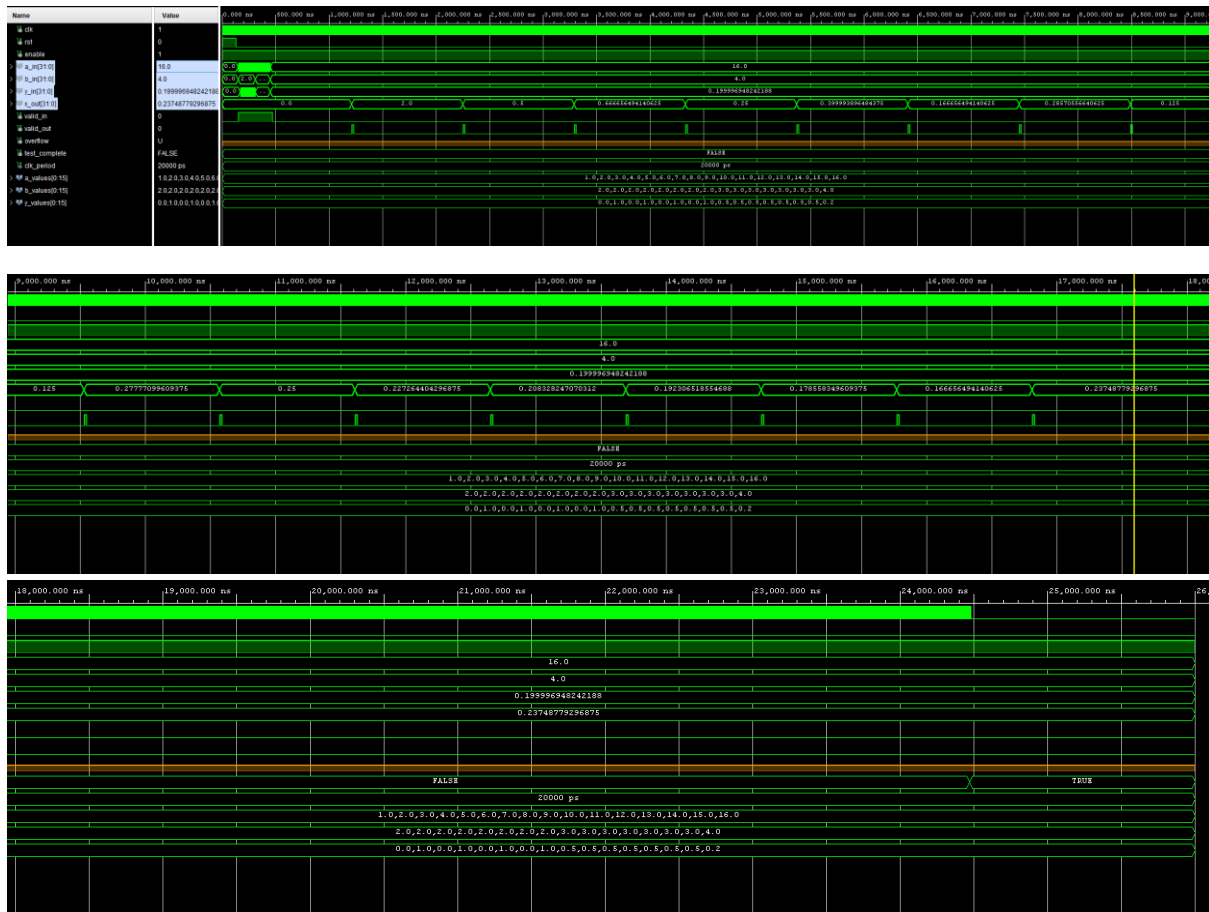


Figure 41: Waveforms of multicycle DIVSUB

Reset will reset the register values before enable turns to 1 and the data is processed and the results come in `x_out` later. There is a valid out for every `x_out` to show it has been validated and there are also waveforms for the testbench values.

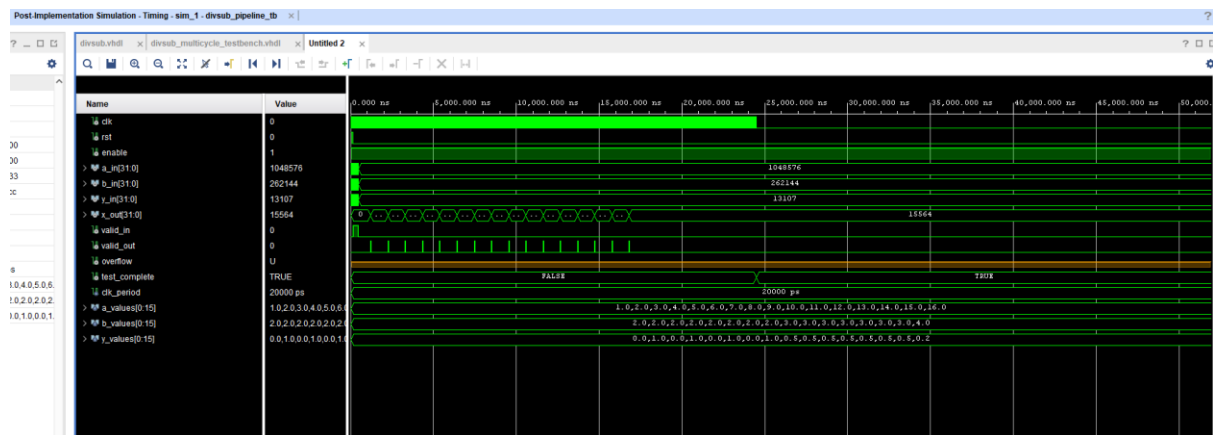


Figure 42: Post implementation timing simulation

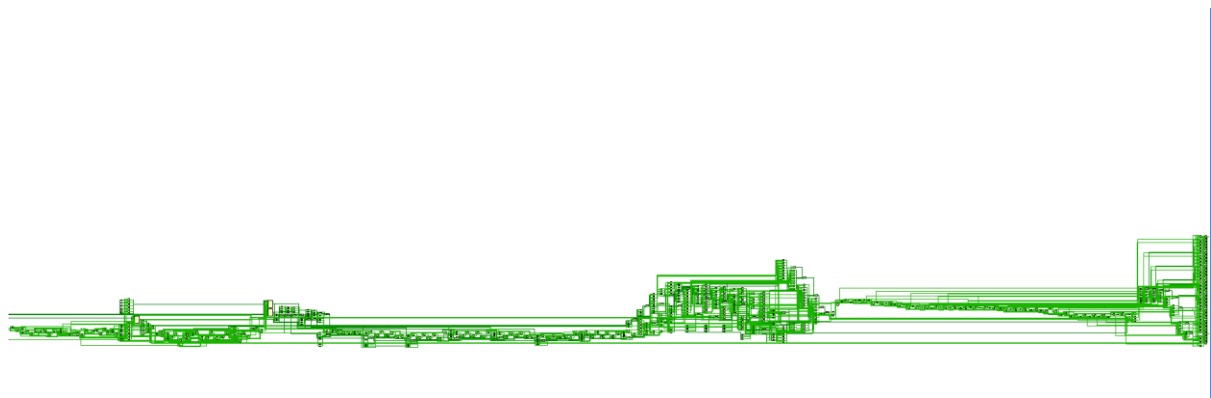


Figure 43: Synthesis multi-cycle DIVSUB

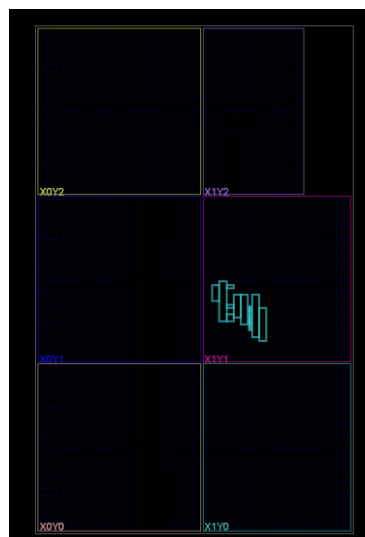


Figure 44: Implementation multi-cycle DIVSUB

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: 0.07 W
Design Power Budget: Not Specified
Process: typical
Power Budget Margin: N/A
Junction Temperature: 25.4°C
 Thermal Margin: 59.6°C (11.9 W)
 Ambient Temperature: 25.0 °C
 Effective θ_{JA} : 5.0°C/W
 Power supplied to off-chip devices: 0 W
 Confidence level: Medium

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

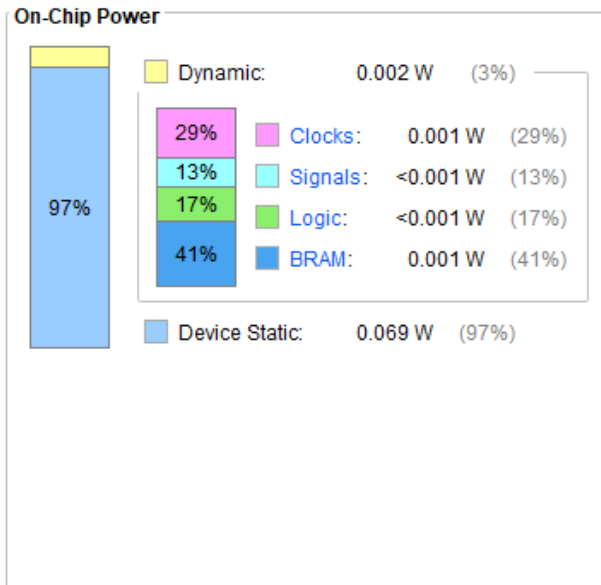


Figure 45: Power multi-cycle DIVSUB

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 11.898 ns	Worst Hold Slack (WHS): 0.150 ns	Worst Pulse Width Slack (WPWS): 9.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 417	Total Number of Endpoints: 417	Total Number of Endpoints: 199

All user specified timing constraints are met.

Figure 46: Post implement Timing report Multi-cycle DIVSUB

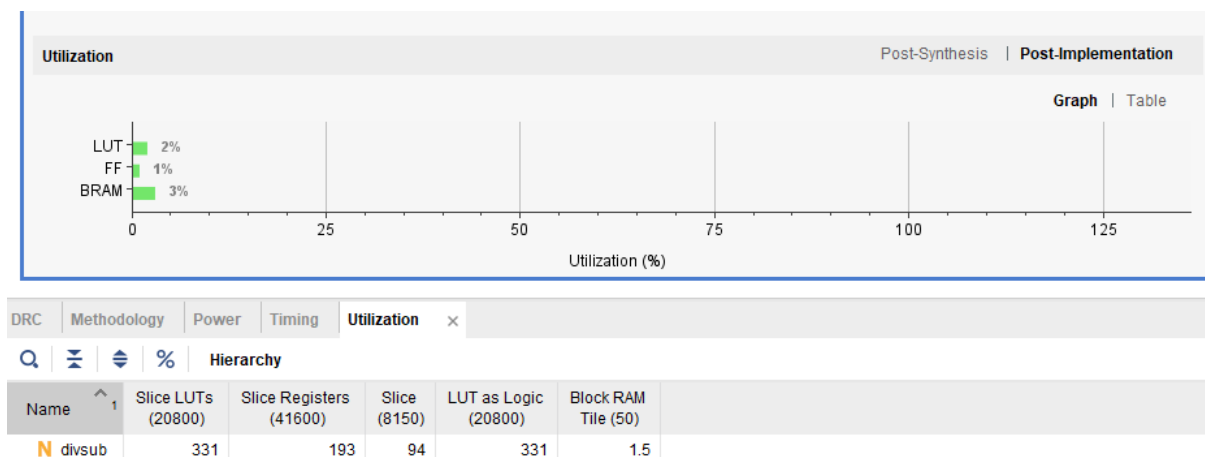


Figure 47: Multi-cycle utilisation (Post implementation and synthesis are the same)

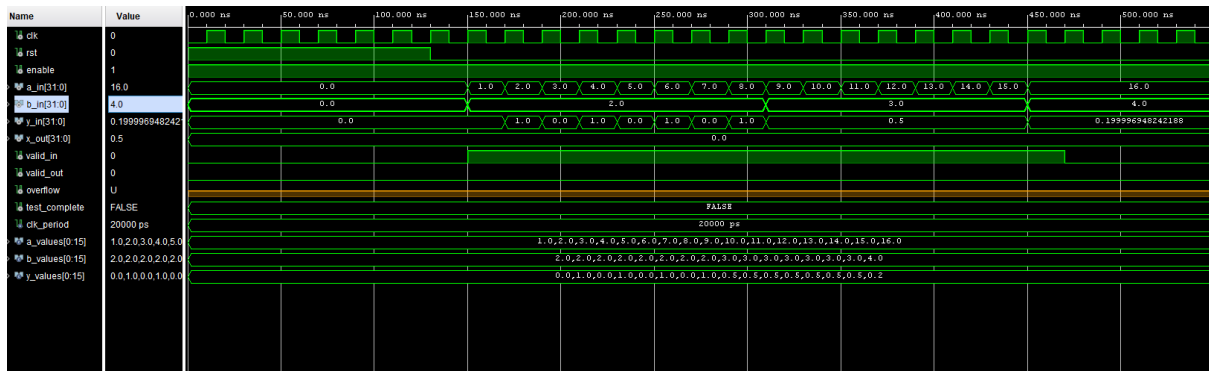
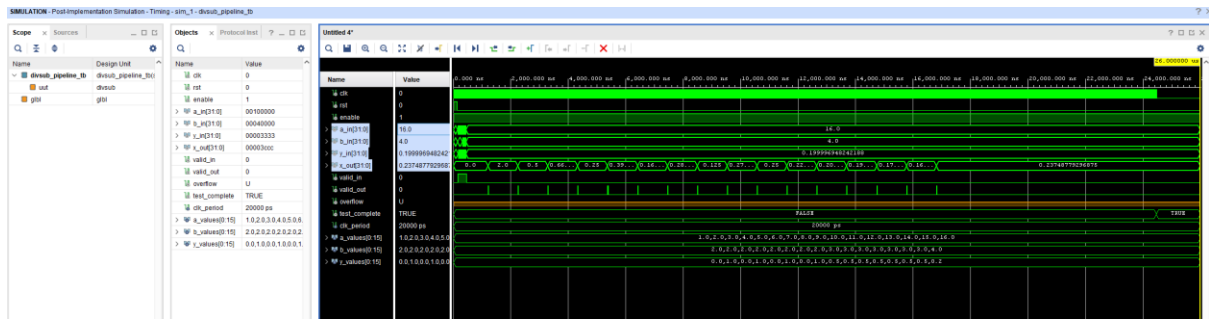


Figure 48: Post implementation timing (second image zoomed)

The reason why a multi-cycle DIVSUB is needed and why IPSP can complete multiplication in one cycle is because of how the arithmetic fundamentally works. When computing multiplication, partial products can be summed in parallel with each other however with division it is sequential fundamentally. Must calculate remainder from quotient and dividend and then subtract. There are multiple stages that cannot be done in parallel.

Systolic Array

Part A

```
1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3. use IEEE.NUMERIC_STD.ALL;
4. use work.fp_pkg.all;
5.
6. entity systolic_band_solver is
7.     port (
8.         clk          : in  std_logic;
9.         rst          : in  std_logic; -- external async reset; synchronized inside
10.        start_solve   : in  std_logic;
11.
12.        -- Matrix inputs (diagonal D and subdiagonal L)
13.        a11, a22, a33, a44 : in  fp32; -- diagonal
14.        a21, a32, a43      : in  fp32; -- subdiagonal (below diagonal)
15.
16.        -- RHS vector
17.        b1, b2, b3, b4 : in  fp32;
18.    );
```

```

19.         -- Solution outputs
20.         x1, x2, x3, x4 : out fp32;
21.
22.         -- Status
23.         solve_done : out std_logic;
24.         solve_busy : out std_logic
25.     );
26. end systolic_band_solver;
27.
28. architecture student_template of systolic_band_solver is
29.
30.     signal ipsp_a_latched : fp32 := FP_ZERO;
31.     signal ipsp_x_latched : fp32 := FP_ZERO;
32.
33.     -----
34.     -- Subcomponents
35.     -----
36.     component divsub
37.     port (
38.         clk      : in  std_logic;
39.         rst      : in  std_logic;
40.         enable   : in  std_logic;
41.         a_in     : in  fp32;
42.         b_in     : in  fp32;
43.         y_in     : in  fp32;
44.         x_out    : out fp32;
45.         valid_in : in  std_logic;
46.         valid_out : out std_logic
47.     );
48. end component;
49.
50.     component ipsp_pipelined
51.     port (
52.         clk      : in  std_logic;
53.         rst      : in  std_logic;
54.         en       : in  std_logic;
55.         a_in     : in  fp32;
56.         x_in     : in  fp32;
57.         y_in     : in  fp32;
58.         a_out    : out fp32;
59.         x_out    : out fp32;
60.         y_out    : out fp32;
61.         valid_in : in  std_logic;
62.         valid_out : out std_logic
63.     );
64. end component;
65.
66.     -----
67.     -- Reset synchronizer (2-FF) and synchronous warmup
68.     -----
69.     attribute ASYNC_REG : string;
70.
71.     signal rst_meta      : std_logic := '1';
72.     signal rst_syncd     : std_logic := '1';
73.     attribute ASYNC_REG of rst_meta  : signal is "TRUE";
74.     attribute ASYNC_REG of rst_syncd : signal is "TRUE";
75.
76.     signal rst_int      : std_logic := '1'; -- fully synchronous internal reset
77.     signal warmup       : unsigned(3 downto 0) := (others => '0');
78.     signal warmup_done  : std_logic := '0';

```

```

79.
80. -----
81. -- Systolic interconnect
82. -----
83. signal y_flow_left : fp32; -- legacy name used elsewhere
84. signal x_flow_right : fp32;
85.
86. type fp_array_4 is array (0 to 3) of fp32;
87. signal a_flow_down : fp_array_4 := (others => FP_ZERO); -- kept for compat
88.
89. -----
90. -- Control signals to blocks
91. -----
92. signal divsub_enable : std_logic := '0';
93. signal divsub_valid_in : std_logic := '0';
94. signal divsub_valid_out : std_logic;
95. signal divsub_valid_out_reg : std_logic := '0'; -- Registered version
96. signal divsub_a_in : fp32 := FP_ZERO;
97. signal divsub_b_in : fp32 := FP_ZERO;
98.
99. signal ipsp_enable : std_logic := '0';
100. signal ipsp_valid_in : std_logic := '0';
101. signal ipsp_valid_out : std_logic;
102. signal ipsp_valid_out_reg : std_logic := '0'; -- Registered version
103.
104. -- Gated (masked) enables/valids during warmup
105. signal divsub_enable_g : std_logic;
106. signal ipsp_enable_g : std_logic;
107. signal ipsp_valid_in_g : std_logic;
108.
109. -----
110. -- Local storage for matrix and RHS
111. -----
112. type fp_diag_array is array (1 to 4) of fp32;
113. type fp_sub_array is array (1 to 3) of fp32;
114. type fp_rhs_array is array (1 to 4) of fp32;
115.
116. signal diagonal_elements : fp_diag_array := (others => FP_ZERO);
117. signal subdiag_elements : fp_sub_array := (others => FP_ZERO);
118. signal rhs_elements : fp_rhs_array := (others => FP_ZERO);
119.
120. -----
121. -- Captured solutions with registered outputs
122. -----
123. signal x1_captured, x2_captured, x3_captured, x4_captured : fp32 := FP_ZERO;
124. signal x1_reg, x2_reg, x3_reg, x4_reg : fp32 := FP_ZERO; -- Registered outputs
125.
126. -----
127. -- Controller FSM
128. -----
129. type state_t is (IDLE, LOAD_DATA, SYSTOLIC_PIPELINE, DONE);
130. signal state : state_t := IDLE;
131. signal cycle_counter : integer range 0 to 255 := 0;
132. signal pipeline_stage : integer range 0 to 7 := 0;
133. signal stage_complete : std_logic_vector(1 to 7) := (others => '0');
134.
135. signal y_flow_left_1: fp32 := FP_ZERO;
136.
137. -----
138. -- Robust y handling for DIVSUB

```

```

139. -----
140. signal y_flow_acc : fp32 := FP_ZERO; -- registered y used by divsub
141. signal y_flow_next : fp32 := FP_ZERO; -- raw y from ipsp
142.
143. -----
144. -- Stable IPSP command regs (loaded 1 clk before valid pulse)
145. -----
146. signal ipsp_a_cmd : fp32 := FP_ZERO;
147. signal ipsp_x_cmd : fp32 := FP_ZERO;
148.
149. -----
150. -- Registered data path for stability
151. -----
152. signal x_flow_right_reg : fp32 := FP_ZERO;
153.
154. begin
155. -----
156. -- Reset sync (no async PRE/CLR inside design) and synchronous warmup
157. -----
158. process(clk)
159. begin
160.     if rising_edge(clk) then
161.         rst_meta <= rst; -- sample external reset
162.         rst_syncd <= rst_meta; -- second stage
163.     end if;
164. end process;
165. rst_int <= rst_syncd;
166.
167. process(clk)
168. begin
169.     if rising_edge(clk) then
170.         if rst_int = '1' then
171.             warmup <= (others => '0');
172.             warmup_done <= '0';
173.         else
174.             if warmup_done = '0' then
175.                 warmup <= warmup + 1;
176.                 if warmup = "1111" then -- Extended to 15 quiet cycles after reset
for timing simulation
177.                     warmup_done <= '1';
178.                 end if;
179.             end if;
180.         end if;
181.     end if;
182. end process;
183.
184. -----
185. -- Register valid outputs from components for timing stability
186. -----
187. process(clk)
188. begin
189.     if rising_edge(clk) then
190.         if rst_int = '1' then
191.             divsub_valid_out_reg <= '0';
192.             ipsp_valid_out_reg <= '0';
193.             x_flow_right_reg <= FP_ZERO;
194.         else
195.             divsub_valid_out_reg <= divsub_valid_out;
196.             ipsp_valid_out_reg <= ipsp_valid_out;
197.             x_flow_right_reg <= x_flow_right;

```

```

198.         end if;
199.     end if;
200. end process;
201.
202. -----
203. -- Instances
204. -----
205. divsub_proc: divsub
206.     port map (
207.         clk      => clk,
208.         rst       => rst_int,
209.         enable    => divsub_enable_g,
210.         a_in      => divsub_a_in,
211.         b_in      => divsub_b_in,
212.         y_in      => y_flow_acc,    -- registered; never 'X'
213.         x_out     => x_flow_right,
214.         valid_in  => divsub_valid_in,
215.         valid_out => divsub_valid_out
216.     );
217.
218. ipsp_proc: ipsp_pipelined
219.     port map (
220.         clk      => clk,
221.         rst       => rst_int,
222.         en        => ipsp_enable_g,
223.         a_in      => ipsp_a_latched,
224.         x_in      => ipsp_x_latched,
225.         y_in      => FP_ZERO,
226.         a_out     => open,
227.         x_out     => open,
228.         y_out     => y_flow_next,
229.         valid_in  => ipsp_valid_in_g,
230.         valid_out => ipsp_valid_out
231.     );
232.
233. -----
234. -- IPSP command latch
235. -----
236. ipsp_cmd_latch: process(clk)
237. begin
238.     if rising_edge(clk) then
239.         if rst_int = '1' then
240.             ipsp_a_latched <= FP_ZERO;
241.             ipsp_x_latched <= FP_ZERO;
242.         elsif ipsp_valid_in = '1' then
243.             ipsp_a_latched <= ipsp_a_cmd;
244.             ipsp_x_latched <= ipsp_x_cmd;
245.         end if;
246.     end if;
247. end process;
248.
249. -- gate enables/valid during warmup to avoid DSP/BRAM timing notifiers
250. divsub_enable_g <= divsub_enable and warmup_done;
251. ipsp_enable_g   <= ipsp_enable   and warmup_done;
252. ipsp_valid_in_g <= ipsp_valid_in and warmup_done;
253.
254. -- legacy name drive
255. y_flow_left    <= y_flow_acc;
256. y_flow_left_1  <= FP_ZERO;
257.

```



```

258. -----
259. -- y_latch process with priority handling
260. -----
261. y_latch: process(clk)
262. begin
263.     if rising_edge(clk) then
264.         if rst_int = '1' then
265.             y_flow_acc <= FP_ZERO;
266.         else
267.             -- Priority: clear first, then capture
268.             if divsub_valid_out_reg = '1' then
269.                 y_flow_acc <= FP_ZERO;
270.             elsif ipsp_valid_out_reg = '1' then
271.                 y_flow_acc <= y_flow_next;
272.             end if;
273.         end if;
274.     end if;
275. end process;
276.
277. -----
278. -- Stage completion detector (synchronous reset)
279. -----
280. stage_coordinator: process(clk)
281. begin
282.     if rising_edge(clk) then
283.         if rst_int = '1' then
284.             stage_complete <= (others => '0');
285.         else
286.             case pipeline_stage is
287.                 when 1 => stage_complete(1) <= divsub_valid_out_reg;
288.                 when 2 => stage_complete(2) <= ipsp_valid_out_reg;
289.                 when 3 => stage_complete(3) <= divsub_valid_out_reg;
290.                 when 4 => stage_complete(4) <= ipsp_valid_out_reg;
291.                 when 5 => stage_complete(5) <= divsub_valid_out_reg;
292.                 when 6 => stage_complete(6) <= ipsp_valid_out_reg;
293.                 when 7 => stage_complete(7) <= divsub_valid_out_reg;
294.                 when others => stage_complete <= (others => '0');
295.             end case;
296.         end if;
297.     end if;
298. end process;
299.
300. -----
301. -- Controller FSM with IPSP cmd regs & one-cycle valid pulses (sync reset)
302. -----
303. controller: process(clk)
304. begin
305.     if rising_edge(clk) then
306.         if rst_int = '1' then
307.             diagonal_elements <= (others => FP_ZERO);
308.             subdiag_elements <= (others => FP_ZERO);
309.             rhs_elements <= (others => FP_ZERO);
310.
311.             x1_captured <= FP_ZERO;
312.             x2_captured <= FP_ZERO;
313.             x3_captured <= FP_ZERO;
314.             x4_captured <= FP_ZERO;
315.
316.             state <= IDLE;
317.             cycle_counter <= 0;

```

```

318.         pipeline_stage <= 0;
319.
320.         divsub_enable  <= '0';
321.         divsub_valid_in <= '0';
322.         ipsp_enable    <= '0';
323.         ipsp_valid_in  <= '0';
324.
325.         ipsp_a_cmd <= FP_ZERO;
326.         ipsp_x_cmd <= FP_ZERO;
327.
328.     else
329.         divsub_valid_in <= '0';
330.         ipsp_valid_in  <= '0';
331.
332.     case state is
333.
334.         when IDLE =>
335.             divsub_enable <= '0';
336.             ipsp_enable  <= '0';
337.             if start_solve = '1' then
338.                 state <= LOAD_DATA;
339.             end if;
340.
341.         when LOAD_DATA =>
342.             diagonal_elements(1) <= a11;
343.             diagonal_elements(2) <= a22;
344.             diagonal_elements(3) <= a33;
345.             diagonal_elements(4) <= a44;
346.             subdiag_elements(1)  <= a21;
347.             subdiag_elements(2)  <= a32;
348.             subdiag_elements(3)  <= a43;
349.             rhs_elements(1)      <= b1;
350.             rhs_elements(2)      <= b2;
351.             rhs_elements(3)      <= b3;
352.             rhs_elements(4)      <= b4;
353.             pipeline_stage <= 1;
354.             cycle_counter  <= 0;
355.             state <= SYSTOLIC_PIPELINE;
356.
357.         when SYSTOLIC_PIPELINE =>
358.
359.             case pipeline_stage is
360.
361.                 when 1 => -- x1
362.                     if cycle_counter = 0 then
363.                         divsub_enable <= '1';
364.                         divsub_a_in  <= diagonal_elements(1);
365.                         divsub_b_in  <= rhs_elements(1);
366.                         cycle_counter <= 1;
367.                     elsif cycle_counter = 1 then
368.                         divsub_valid_in <= '1';
369.                         cycle_counter <= 2;
370.                     elsif divsub_valid_out_reg = '1' then
371.                         divsub_enable <= '0';
372.                         x1_captured <= x_flow_right_reg;
373.                         pipeline_stage <= 2;
374.                         cycle_counter <= 0;
375.                     end if;
376.
377.                 when 2 => -- y1

```

```

378.         if cycle_counter = 0 then
379.             ipsp_enable <= '1';
380.             ipsp_a_cmd  <= subdiag_elements(1);
381.             ipsp_x_cmd  <= x1_captured;
382.             cycle_counter <= 1;
383.         elsif cycle_counter = 1 then
384.             ipsp_valid_in <= '1';
385.             cycle_counter <= 2;
386.         elsif ipsp_valid_out_reg = '1' then
387.             ipsp_enable <= '0';
388.             pipeline_stage <= 3;
389.             cycle_counter <= 0;
390.         end if;
391.
392.     when 3 =>    -- x2
393.         if cycle_counter = 0 then
394.             divsub_enable <= '1';
395.             divsub_a_in <= diagonal_elements(2);
396.             divsub_b_in <= rhs_elements(2);
397.             cycle_counter <= 1;
398.         elsif cycle_counter = 1 then
399.             divsub_valid_in <= '1';
400.             cycle_counter <= 2;
401.         elsif divsub_valid_out_reg = '1' then
402.             divsub_enable <= '0';
403.             x2_captured <= x_flow_right_reg;
404.             pipeline_stage <= 4;
405.             cycle_counter <= 0;
406.         end if;
407.
408.     when 4 =>    -- y2
409.         if cycle_counter = 0 then
410.             ipsp_enable <= '1';
411.             ipsp_a_cmd  <= subdiag_elements(2);
412.             ipsp_x_cmd  <= x2_captured;
413.             cycle_counter <= 1;
414.         elsif cycle_counter = 1 then
415.             ipsp_valid_in <= '1';
416.             cycle_counter <= 2;
417.         elsif ipsp_valid_out_reg = '1' then
418.             ipsp_enable <= '0';
419.             pipeline_stage <= 5;
420.             cycle_counter <= 0;
421.         end if;
422.
423.     when 5 =>    -- x3
424.         if cycle_counter = 0 then
425.             divsub_enable <= '1';
426.             divsub_a_in <= diagonal_elements(3);
427.             divsub_b_in <= rhs_elements(3);
428.             cycle_counter <= 1;
429.         elsif cycle_counter = 1 then
430.             divsub_valid_in <= '1';
431.             cycle_counter <= 2;
432.         elsif divsub_valid_out_reg = '1' then
433.             divsub_enable <= '0';
434.             x3_captured <= x_flow_right_reg;
435.             pipeline_stage <= 6;
436.             cycle_counter <= 0;
437.         end if;

```

```

438.
439.         when 6 =>    -- y3
440.             if cycle_counter = 0 then
441.                 ipsp_enable <= '1';
442.                 ipsp_a_cmd  <= subdiag_elements(3);
443.                 ipsp_x_cmd  <= x3_captured;
444.                 cycle_counter <= 1;
445.             elsif cycle_counter = 1 then
446.                 ipsp_valid_in <= '1';
447.                 cycle_counter <= 2;
448.             elsif ipsp_valid_out_reg = '1' then
449.                 ipsp_enable <= '0';
450.                 pipeline_stage <= 7;
451.                 cycle_counter <= 0;
452.             end if;
453.
454.         when 7 =>    -- x4
455.             if cycle_counter = 0 then
456.                 divsub_enable <= '1';
457.                 divsub_a_in <= diagonal_elements(4);
458.                 divsub_b_in <= rhs_elements(4);
459.                 cycle_counter <= 1;
460.             elsif cycle_counter = 1 then
461.                 divsub_valid_in <= '1';
462.                 cycle_counter <= 2;
463.             elsif divsub_valid_out_reg = '1' then
464.                 divsub_enable <= '0';
465.                 x4_captured <= x_flow_right_reg;
466.                 state <= DONE;
467.             end if;
468.
469.         when others => state <= DONE;
470.     end case;
471.
472.     when DONE =>
473.         -- Register the final outputs for timing stability
474.         x1_reg <= x1_captured;
475.         x2_reg <= x2_captured;
476.         x3_reg <= x3_captured;
477.         x4_reg <= x4_captured;
478.         state <= IDLE;
479.
480.     end case;
481. end if;
482. end if;
483. end process;
484.
485. -----
486. -- Outputs with registered values
487. -----
488. x1 <= x1_reg;
489. x2 <= x2_reg;
490. x3 <= x3_reg;
491. x4 <= x4_reg;
492.
493. solve_busy <= '1' when state /= IDLE and state /= DONE else '0';
494. solve_done <= '1' when state = DONE else '0';
495.
496. end student_template;

```

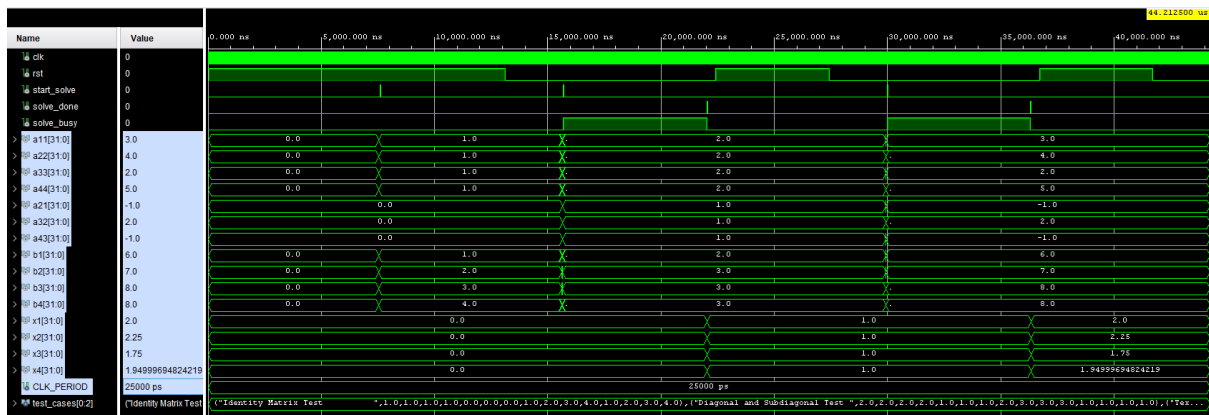


Figure 49: Behavioural sim, cross hatchings are 0

Looking at the waveforms, the testbench performs three distinct examples based on the a and b registers. The start_solve and solve_done are control signals used to monitor. It is the same as valid_in and valid_out on a top level. Solve_busy is a control signal monitoring that the design is currently processing the data which is why it is high as soon as new inputs are added. The arithmetic is done in parallel with x4 being the result. The design resets after every solve_done to isolate each test case and clear each register so there is no data hanging around. It may be hard to tell in the diagram but after every reset all registers return to 0.

Part B

This controller (FSM) solves a matrix vector equation with A and B being known vectors and x being an unknown vector so that $Ax = b$ and it solves for x. It inputs data into a systolic pipeline for computation in a 7-stage process. $X1 = b1 / a11$ using the DIVSUB component. Then y must be calculated using the IPSP component as $y = a21 * x1$ which will be used for the x2 computation as $x2 = (b2 - y) / a22$. This process repeats diagonally down the A matrix until x4 is calculated.

Lines 301 -> to 309 shows how the A matrix is loaded diagonally whilst each stage is annotated in the code showing the 7 steps clearly.

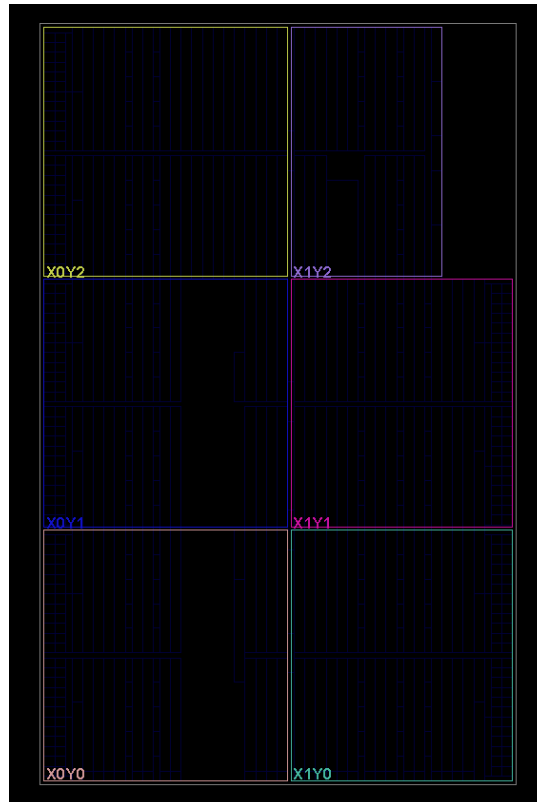


Figure 50: Synthesised design: Systolic Array

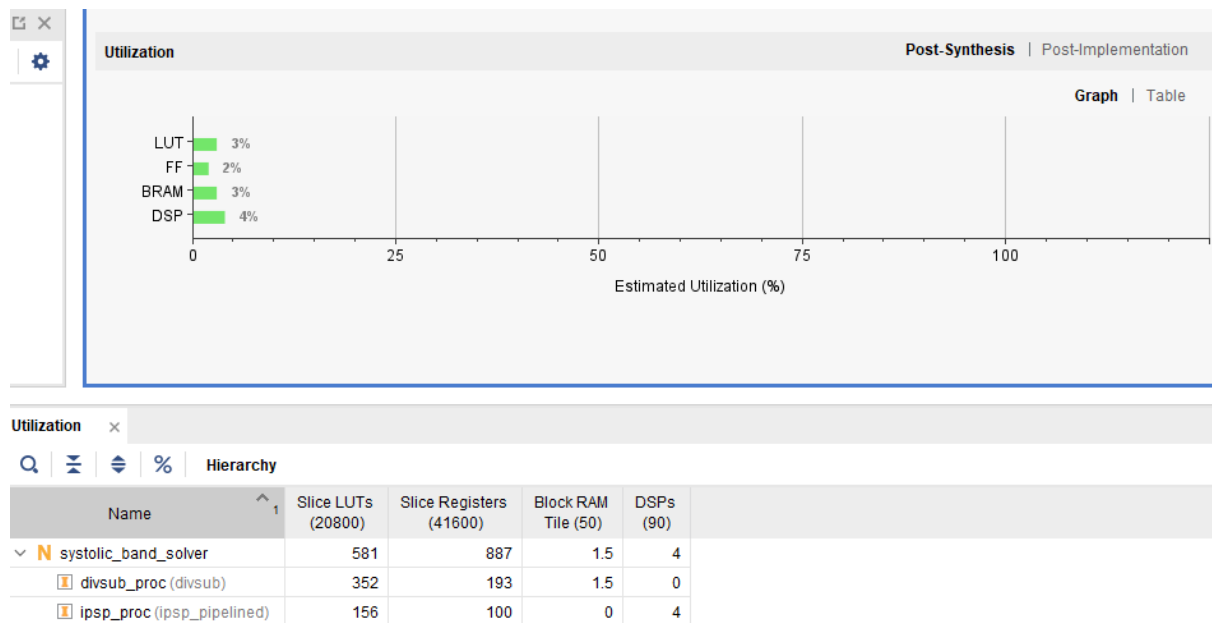


Figure 51: Systolic array utilisation

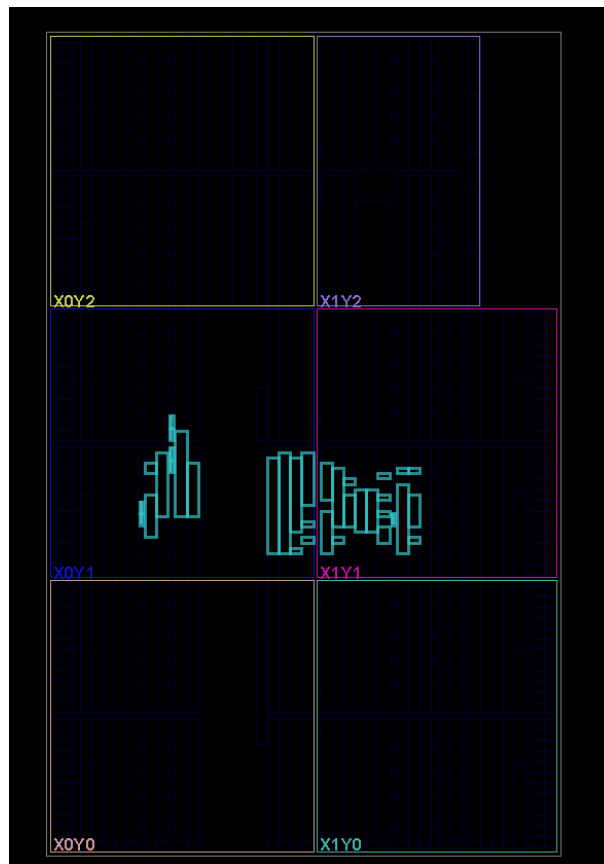


Figure 52: Systolic array implemented design

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: 0.072 W
Design Power Budget: Not Specified
Process: typical
Power Budget Margin: N/A
Junction Temperature: 25.4°C
 Thermal Margin: 59.6°C (11.9 W)
 Ambient Temperature: 25.0 °C
 Effective θ_{JA} : 5.0°C/W
 Power supplied to off-chip devices: 0 W
 Confidence level: Medium

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

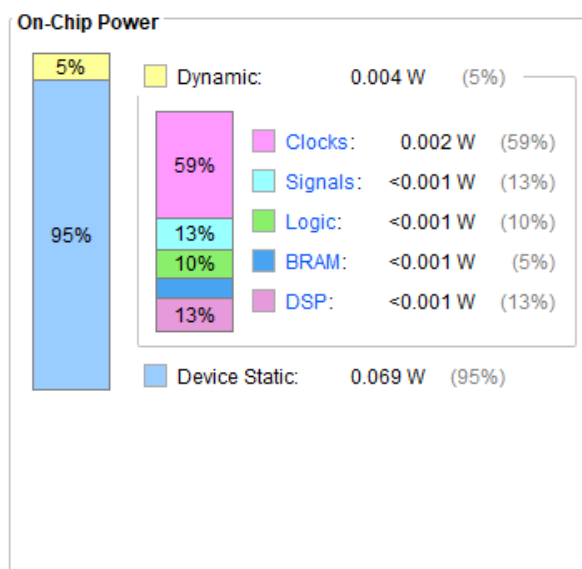


Figure 53: Power usage systolic array

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 35.778 ns	Worst Hold Slack (WHS): 0.070 ns	Worst Pulse Width Slack (WPWS): 24.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 2610	Total Number of Endpoints: 2610	Total Number of Endpoints: 897
All user specified timing constraints are met.		

Figure 54: Timing report

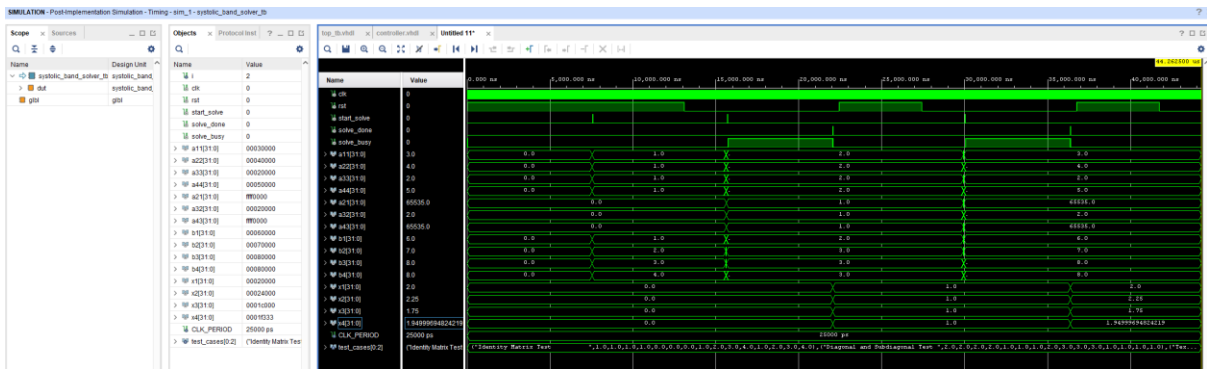


Figure 55: Post implementation timing simulation

Asynchronous Labs

Lab 7

A C-element is used to synchronise the phase of inputs by only having a high output when its inputs are also both high and low when both inputs are low. It can be graphically represented as seen in the figure below.

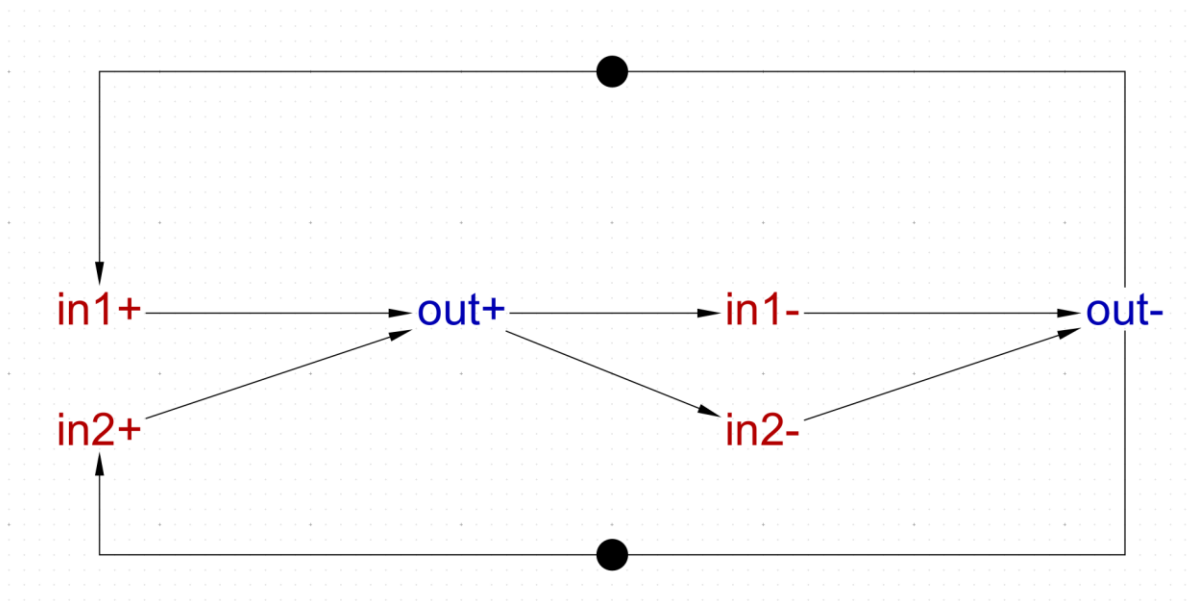


Figure 56: STG model of the 2 input C-element

The DTD (Digital Timing Diagram) shows the behaviour of the STG.

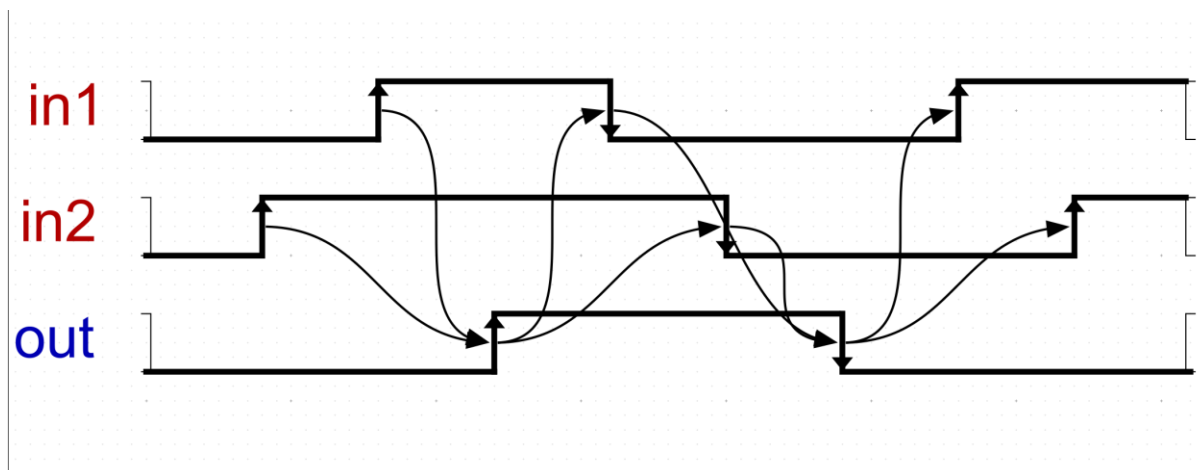


Figure 57: Digital Timing Diagram of the STG

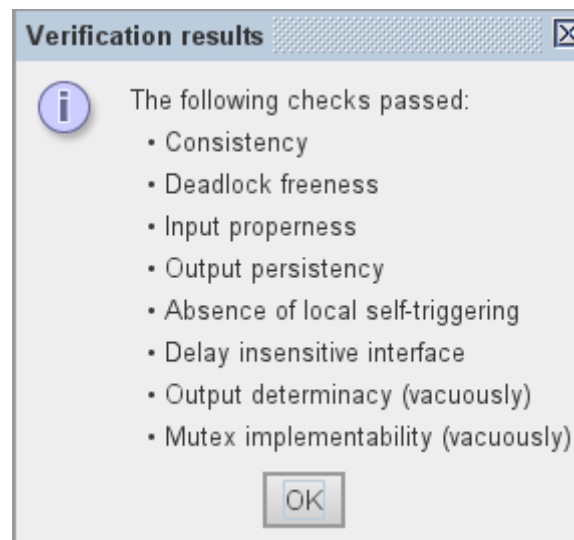


Figure 58: Verification of the STG

Verification is needed to make sure that there are no contradictions etc. in the graph.

There are two mapping methods that can be used to synthesise the STG to a digital circuit. Complex-gate mapping uses complex gates to be customised for the specific behaviour designed, allowing fewer components and an optimal design. However, these complex gates may not exist therefore technology mapping will synthesise the STG using components found in a standard library. If a circuit timing is very important then complex-gate mapping will have a more optimal design especially if designing a specialist circuit. If the circuit does not depend on timing as much or the circuit is desired to be made from readily available components then technology mapping is preferred.

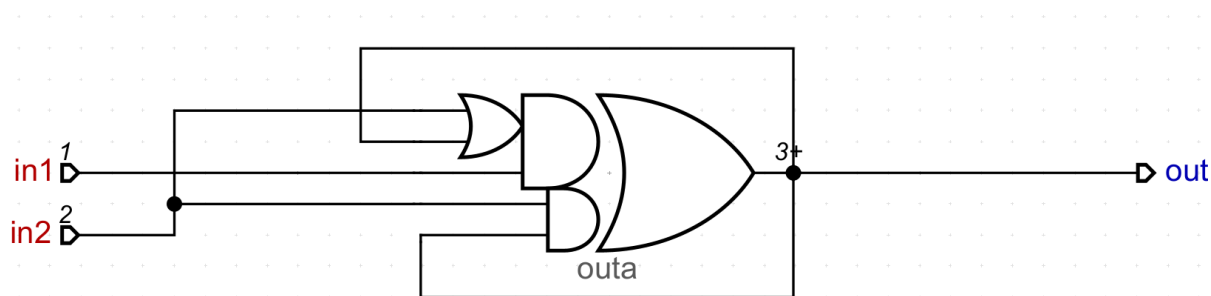


Figure 59: Complex-gate mapping

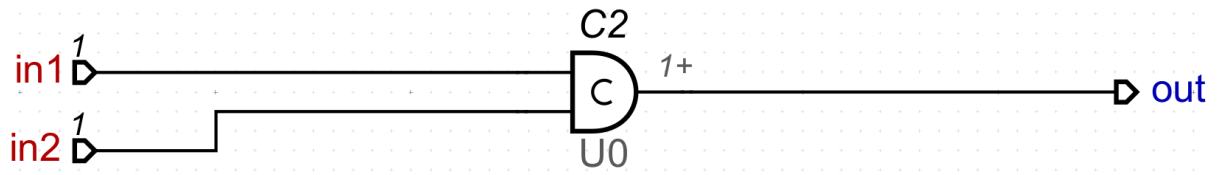


Figure 60: Technology mapping

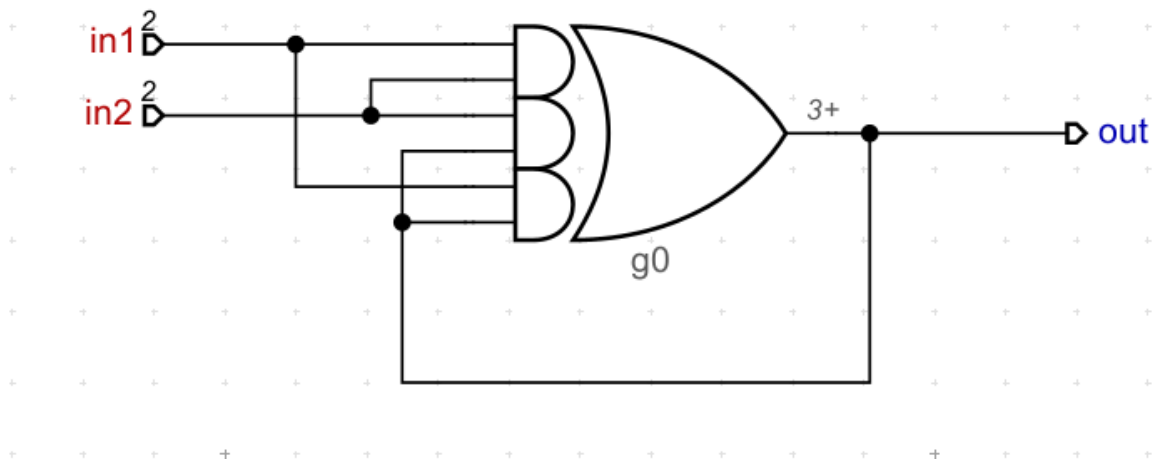


Figure 61: Three 2-input AND gates followed by one 3-input OR gate.

Figure 61 was designed as a digital circuit diagram, and can be converted to a STG.

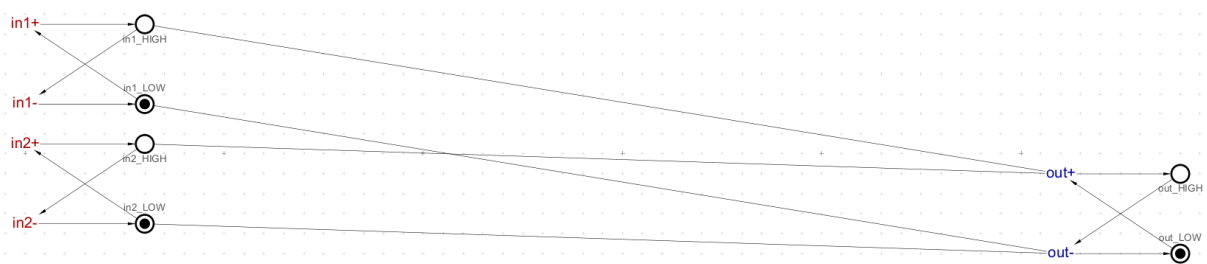


Figure 62: STG Conversion

Undergoing verification, it had output persistency issues. When looking at figure 62, it is clear to see the problem is that the circuit needs to be constrained within a certain environment so that output can finish firing before being deactivated. In this instance, the issue is that in1- or in2- will disable out if activated before out fires.

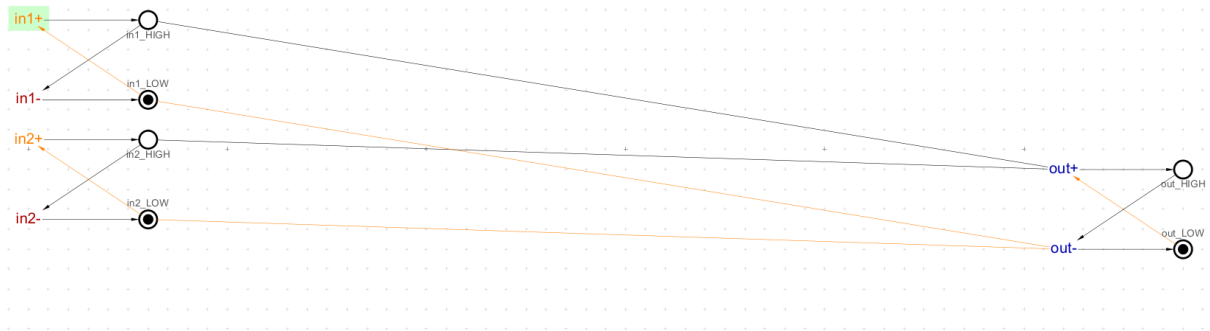


Figure 63: STG without environment

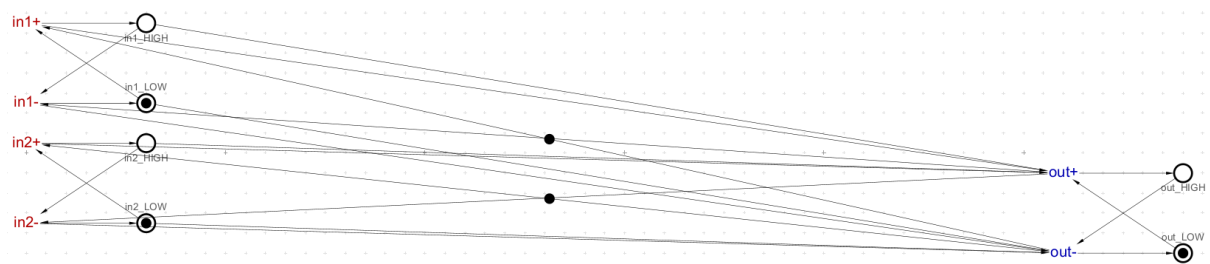


Figure 64: STG under C-element environment

Figure 64 is the STG under the original environment and now that the design is constrained, the output persistency has been corrected. The environment makes sure that out is a priority and only once it has fired then in1- and in2- may fire.

The decomposed circuit verification was perfectly fine however figure 66 with the hazard had output persistency issues.

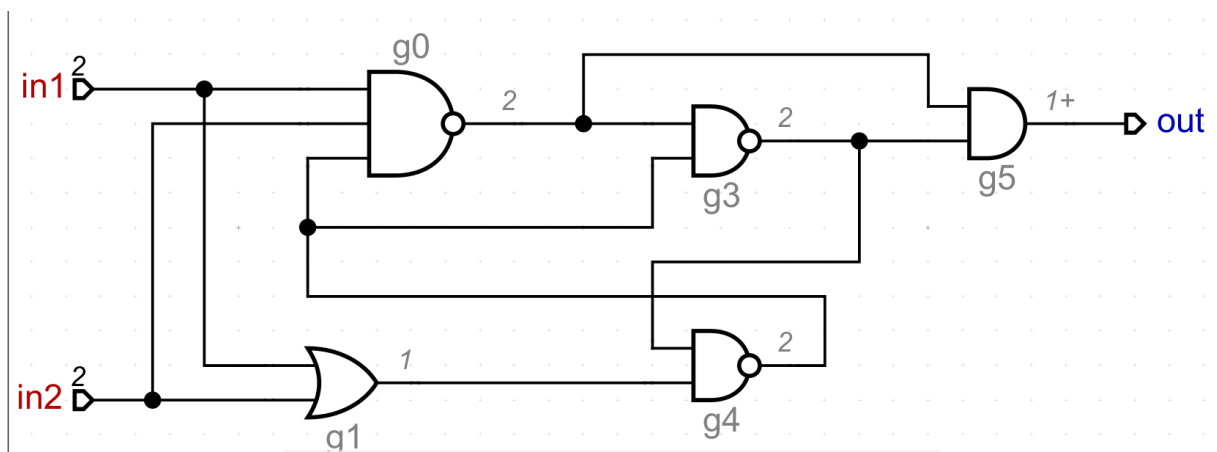


Figure 65: Decomposed circuit

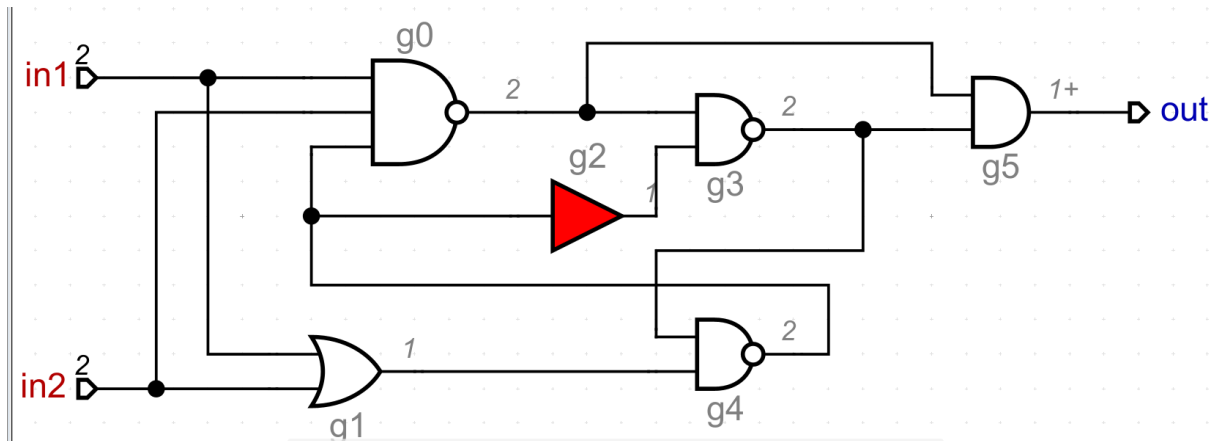


Figure 66: Decomposed circuit with hazard

Namely the hazard circuit is that g0 fires a second time which disables g3 and out before they can finish firing. Due to the added buffer in the circuit, there is a delay mismatch in the branches so that g0 firing is received at different times.

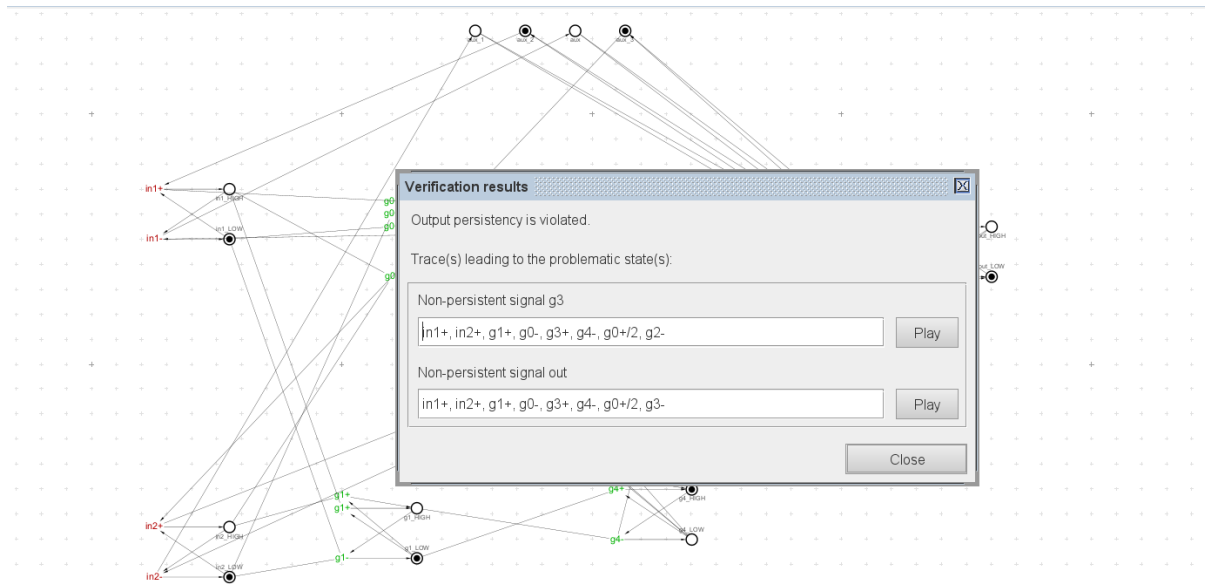


Figure 67: Output persistency issues due to hazard

Lab 8

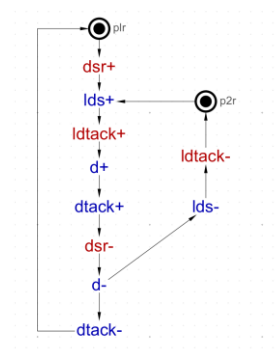


Figure 68: VME read controller

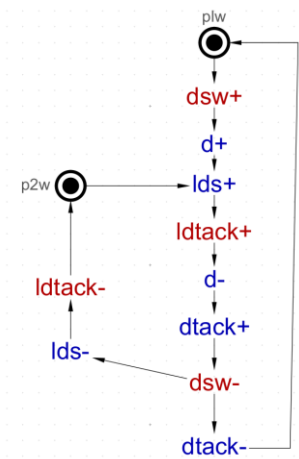


Figure 69: VME write controller

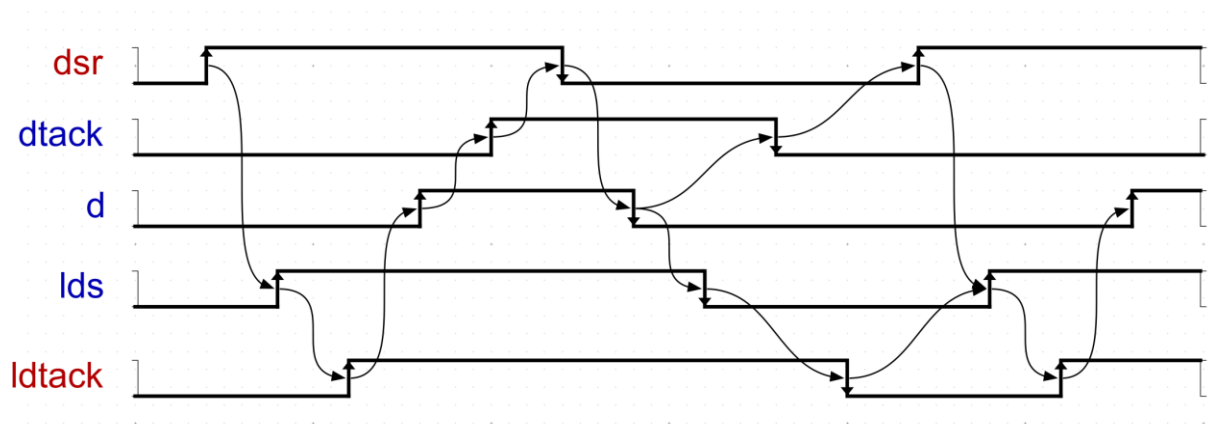


Figure 70: Read validation

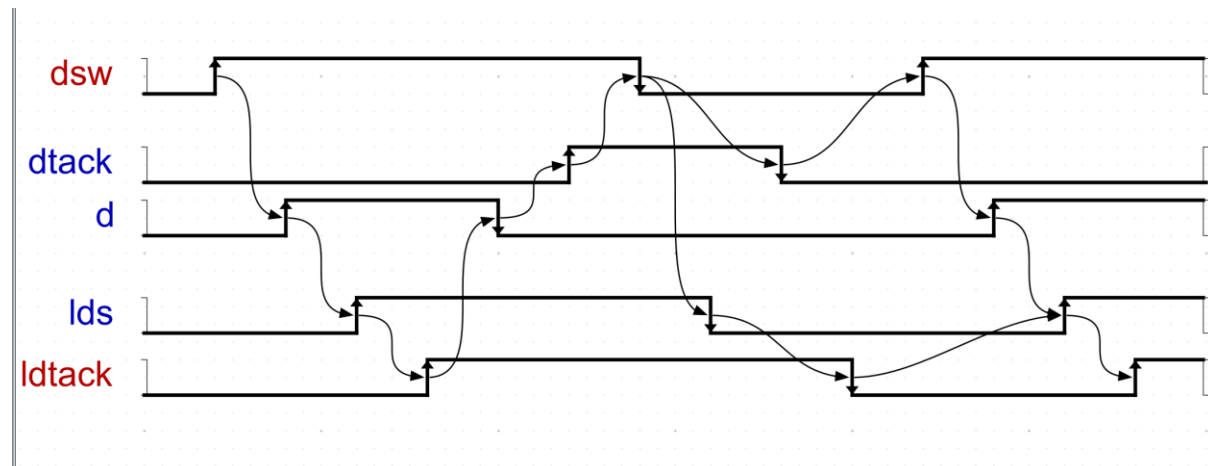


Figure 71: Write validation

Both operations are verified; they can now be merged.

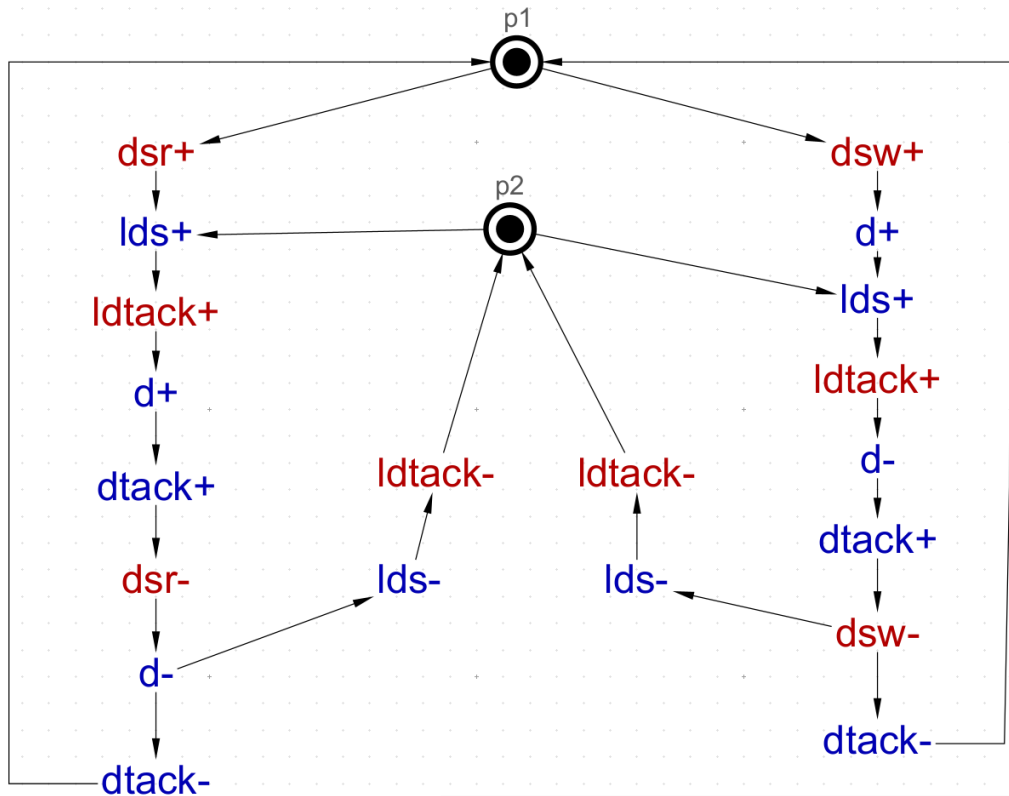


Figure 72: Read-write controller

Since they both share some common values, this can be simplified.

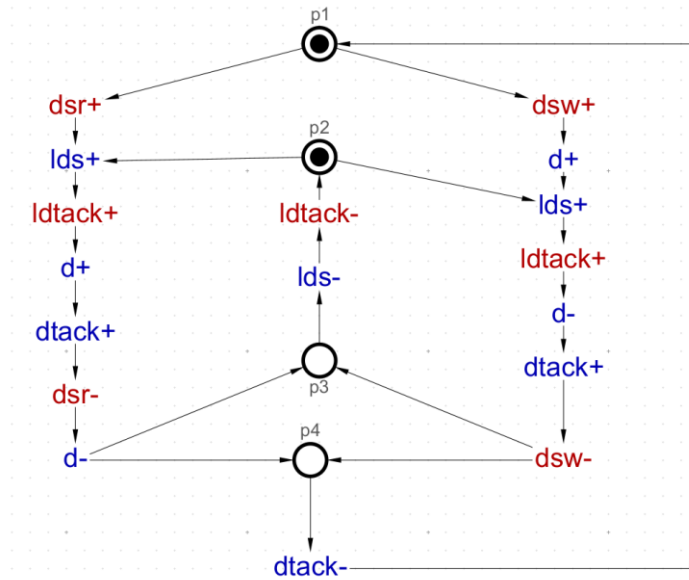


Figure 73: Simplified read-write controller

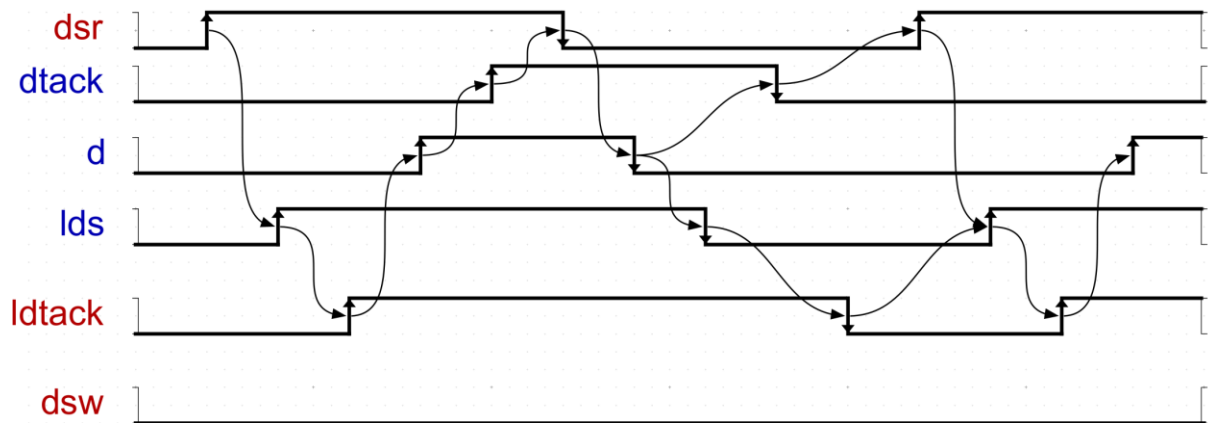


Figure 74: Read-write validation (read)

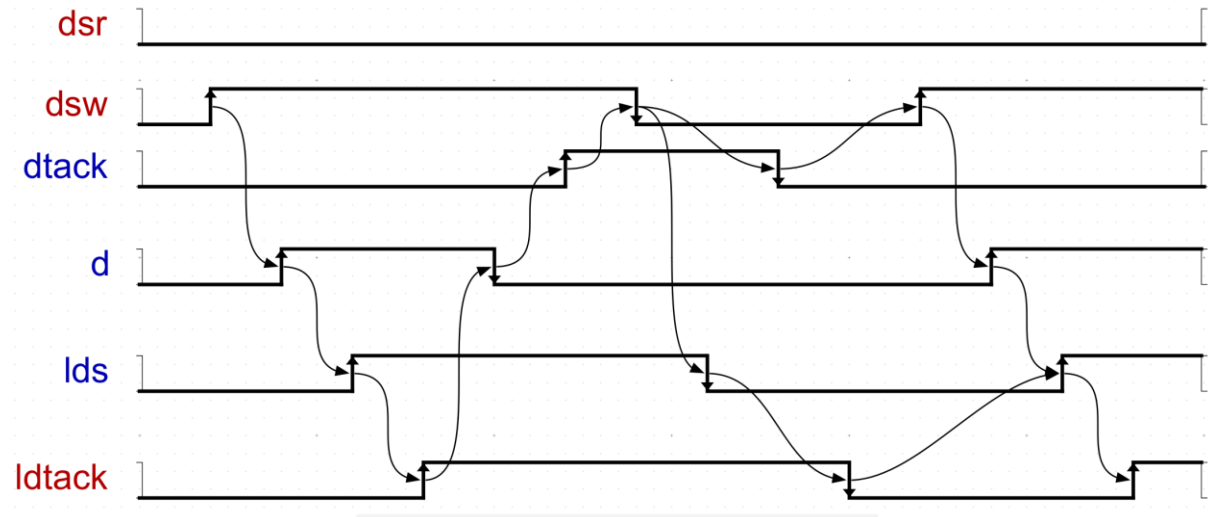


Figure 75: Read-write validation (write)

Figure 73 passes verification. The reason for merging them is so that there is one controller that can handle both reading and writing operations. This is beneficial as now less elements are needed as they share and can have better timing with one another. The detriment is that if a certain element fails then both the reading and writing operations can fail due to one issue.

The read-write controller encounters a CSC issues. This is because it will reach a point where all the values for all signals will be the same with different outputs. In this case, it cannot distinguish between read and writing operations. To fix this, states can be removed so that unreachable states are removed, which is a benefit as it reduces the size- the issue with this is that conflicts usually end up in reachable states, so this is not an option. Internal signals may be added to distinguish between conflicting states as seen in figure 76.

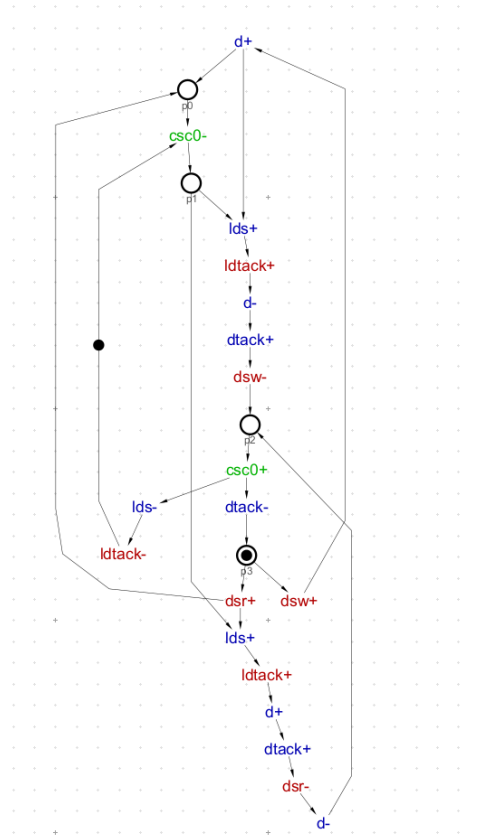


Figure 76: CSC Verification

This can be a detriment as it adds an extra signal which can increase timing, power consumption and a greater overhead however the benefit is that it will work all the time to distinguish between the states. A final method is to redesign the gate synthesis so that the internal nodes can distinguish between states. The benefit is that it can be customised for a particular design however the detriment is that it requires complex gates which are not readily available.

Now that the CSC conflict has been resolved, the circuit may be synthesised. Figure 77 obviously has output persistency until under the correct STG environment which then is fully verified. However this uses complex gates so using technology mapping, a more realistic design can be seen.

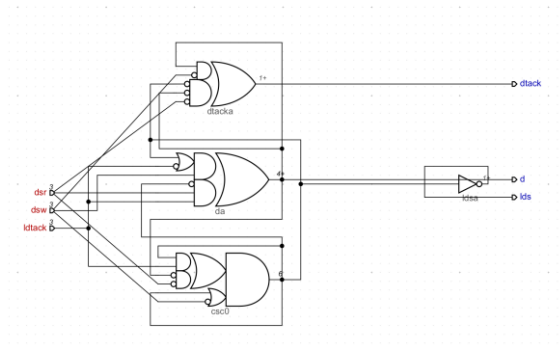


Figure 77: Synthesised circuit using complex-gate mapping

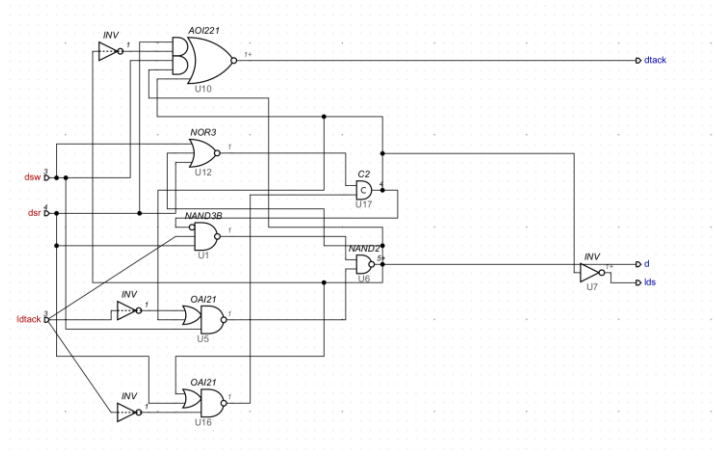


Figure 78: Synthesised circuit using technology mapping

Figure 78 is also fully verified in the environment of the simplified STG and conforms to the STG.

```

1  // Verilog netlist generated by Workcraft 3
2  // * Workcraft version: 3.5.3
3  // * Operating system: Windows 11 amd64
4  // * Creation timestamp: 2026-01-07T19:24:23.035Z
5  // * Design title: Untitled
6  // * JavaScript command: framework.exportWork(workspaceEntry, 'stg-vme-read_write_simplified_verilog.v', 'Verilog')
7  module Untitled (dsr, dsw, ldtack, d, lds, dtack);
8      input dsr, dsw, ldtack;
9      output d, lds, dtack;
10     wire U1_ON, IN_BUBBLE3_ON, U5_ON, IN_BUBBLE9_ON, U12_ON, IN_BUBBLE14_ON, U16_ON, U17_Q;
11
12     NAND3B U1 (.ON(U1_ON), .AN(U17_Q), .B(ldtack), .C(dsr));
13     // This inverter should have a short delay
14     INV IN_BUBBLE3 (.ON(IN_BUBBLE3_ON), .I(ldtack));
15     OAI21 U5 (.ON(U5_ON), .A1(IN_BUBBLE3_ON), .A2(U17_Q), .B(dsw));
16     NAND2 U6 (.ON(d), .A(U1_ON), .B(U5_ON));
17     INV U7 (.ON(lds), .I(U17_Q));
18     // This inverter should have a short delay
19     INV IN_BUBBLE9 (.ON(IN_BUBBLE9_ON), .I(d));
20     AOI221 U10 (.ON(dtack), .A1(dsr), .A2(IN_BUBBLE9_ON), .B1(dsw), .B2(d), .C(U17_Q));
21     NOR3 U12 (.ON(U12_ON), .A(dsw), .B(d), .C(dsr));
22     // This inverter should have a short delay
23     INV IN_BUBBLE14 (.ON(IN_BUBBLE14_ON), .I(ldtack));
24     OAI21 U16 (.ON(U16_ON), .A1(d), .A2(dsr), .B(IN_BUBBLE14_ON));
25     C2 U17 (.Q(U17_Q), .A(U12_ON), .B(U16_ON));
26
27     // signal values at the initial state:
28     // IN_BUBBLE14_ON IN_BUBBLE3_ON IN_BUBBLE9_ON U12_ON U16_ON U17_Q U1_ON U5_ON !d !dsr !dsw !dtack !lds !ldtack
29 endmodule
30

```

Figure 79: Exported Verilog

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity Read_write is
5  port (
6      dsr : in STD_LOGIC;
7      dsw : in STD_LOGIC;
8      ldtack : in STD_LOGIC;
9      d : out STD_LOGIC;
10     lds : out STD_LOGIC;
11     dtack : out STD_LOGIC;
12 );
13 end entity Read_write;
14
15 architecture structural of Read_write is
16     -- Internal wiring signals between gates
17     signal U1_ON : STD_LOGIC;
18     signal IN_BUBBLE3_ON : STD_LOGIC;
19     signal U5_ON : STD_LOGIC;
20     signal IN_BUBBLE9_ON : STD_LOGIC;
21     signal U12_ON : STD_LOGIC;
22     signal IN_BUBBLE14_ON : STD_LOGIC;
23     signal U16_ON : STD_LOGIC;
24     signal U17_Q : STD_LOGIC;
25
26 begin
27     -- NAND with inverted first input: !(U17_Q & ldtack & dsr)
28     U1_ON <= not ((not U17_Q) and ldtack and dsr);
29
30     -- Fast inverter on ldtack signal
31     IN_BUBBLE3_ON <= not ldtack;
32
33     -- OR-AND-INVERT gate: !((IN_BUBBLE3_ON OR U17_Q) & dsd)
34     U5_ON <= not ((IN_BUBBLE3_ON or U17_Q) and dsd);
35
36     -- NAND gate producing main output d: !(U1_ON & U5_ON)
37     d <= not (U1_ON and U5_ON);
38
39     -- Inverter producing lds output: !U17_Q
40     lds <= not U17_Q;
41
42     -- Fast inverter on d signal (feedback path)
43     IN_BUBBLE9_ON <= not d;
44
45     -- AND-OR-INVERT gate: !((dsr & IN_BUBBLE9_ON) OR (dsd & d) OR U17_Q)
46     dtack <= not ((dsr and IN_BUBBLE9_ON) or (dsd and d) or U17_Q);
47
48     -- NOR gate: !(dsd OR d OR dsr)
49     U12_ON <= not (dsd or d or dsr);
50
51     -- Fast inverter on ldtack (another instance)
52     IN_BUBBLE14_ON <= not ldtack;
53
54     -- OR-AND-INVERT gate: !((d OR dsr) & IN_BUBBLE14_ON)
55     U16_ON <= not ((d or dsr) and IN_BUBBLE14_ON);
56
57     -- C-element process: changes only when both inputs agree
58     C_ELEMENT_PROCESS: process(U12_ON, U16_ON)
59     begin
60         if (U12_ON = '1' and U16_ON = '1') then
61             U17_Q <= '1';
62         elsif (U12_ON = '0' and U16_ON = '0') then
63             U17_Q <= '0';
64         -- Else: hold previous state (no else clause = memory behavior)
65         end if;
66     end process C_ELEMENT_PROCESS;
67
68 end architecture structural;

```

Figure 80: VHDL translation of the Verilog

Figure 80 describes the synthesised circuit (figure 78) in terms of VHDL code. Firstly, there is an entity declaration acknowledging inputs and outputs. The architecture describes the behaviour of this circuit and how it is wired. Again, the declaration of internal variables is made and then the behaviour can be described. For example, line 37 is describing how U1_ON and U5_ON are connected to a NAND gate which wires to port d. The C_ELEMENT_PROCESS describes how its logic should behave within the circuit also and therefore the architecture of the circuit has been fully described in VHDL code. VHDL and Verilog are two different methods of describing a circuit. Verilog is shorter so can write up circuits quicker if needed to describe many systems in a limited time frame however Verilog is harder to read at a first glance. VHDL is much better for readability however more typing is required and is more complicated. VHDL is better when working in teams as the behaviour and implementation is much clearer, so less clarification is needed. VHDL can be used in LTspice as although there is no direct implementation, by reading the VHDL description and then manually modelling it into LTspice allows for analogue circuit testing and simulation. It is much better used in Xilinx Vivado where it is the primary language for FPGAs. It is needed to describe circuits but also testbenches that are used to simulate how it will behave under certain

conditions. It can then be seen what registers are mapped to what and the power usage, critical path delay and other timing delays.